



OGC INDOORGML 2.0 PART 2C – SQL ENCODING

STANDARD
Implementation

CANDIDATE SWG DRAFT

Version: 0.5.0

Submission Date: 2026-06-18

Approval Date: 2026-XX-XX

Publication Date: 2026-XX-XX

Editor: Sisi Zlatanova, Zhiyong Wang, Taehoon Kim, Abdoulaye Diakite, Ki-Joune Li

Notice for Drafts: This document is not an OGC Standard. This document is distributed for review and comment. This document is subject to change without notice and may not be referred to as an OGC Standard.

Recipients of this document are invited to submit, with their comments, notification of any relevant patent rights of which they are aware and to provide supporting documentation.

License Agreement

Use of this document is subject to the license agreement at <https://www.ogc.org/license>

Suggested additions, changes and comments on this document are welcome and encouraged. Such suggestions may be submitted using the online change request form on OGC web site: <http://ogc.standardstracker.org/>

Copyright notice

Copyright © 2026 Open Geospatial Consortium

To obtain additional rights of use, visit <https://www.ogc.org/legal>

Note

Attention is drawn to the possibility that some of the elements of this document may be the subject of patent rights. The Open Geospatial Consortium shall not be held responsible for identifying any or all such patent rights.

Recipients of this document are requested to submit, with their comments, notification of any relevant patent claims or other intellectual property rights of which they may be aware that might be infringed by any implementation of the standard set forth in this document, and to provide supporting documentation.

CONTENTS

I. ABSTRACT	vi
II. KEYWORDS	vi
III. PREFACE	vii
IV. SECURITY CONSIDERATIONS	viii
V. SUBMITTING ORGANIZATIONS	ix
VI. SUBMITTERS	ix
1. SCOPE	2
2. CONFORMANCE	4
3. NORMATIVE REFERENCES	6
4. TERMS AND DEFINITIONS	8
5. CONVENTIONS	11
5.1. Symbols (and abbreviated terms)	11
5.2. Identifiers	11
5.3. Use of SQL terminology	11
5.4. Geometry storage	12
5.5. Complex DataType storage	13
5.6. Code list storage	13
5.7. Graph and route associations	13
5.8. Table and column naming	14
6. OVERVIEW OF THE SQL ENCODING	16
6.1. Design Principle	16
7. SQL ENCODING OF THE CORE MODULE	20
7.1. IndoorFeatures	21
7.2. ThematicLayer	21
7.3. PrimalSpaceLayer	23
7.4. CellSpace	23
7.5. CellBoundary	25
7.6. DualSpaceLayer	27

7.7. Node	28
7.8. Edge	29
7.9. InterLayerConnection	30
7.10. ExternalReferenceType	32
8. SQL ENCODING OF THE NAVIGATION MODULE	34
8.1. NavigableSpace	34
8.2. GeneralSpace	35
8.3. TransferSpace	36
8.4. NonNavigableSpace	36
8.5. ObjectSpace	37
8.6. NonNavigableBoundary	38
8.7. NavigableBoundary	38
8.8. Route	39
ANNEX A (NORMATIVE) CONFORMANCE CLASS ABSTRACT TEST SUITE	41
A.1. Conformance Class Core	41
A.2. Conformance Class Navigation	45
ANNEX B (NORMATIVE) SCHEMAS	49
B.1. SQL Schema of IndoorGML Core	49
B.2. SQL Schema of IndoorGML Navigation	56
ANNEX C (NORMATIVE) CORE CODE LIST TYPES	62
ANNEX D (NORMATIVE) NAVIGATION CODE LIST TYPES	64
ANNEX E (INFORMATIVE) REVISION HISTORY	67
BIBLIOGRAPHY	70

LIST OF TABLES

Table 1	vi
Table 2	4
Table 3 – Symbols and abbreviated terms	11
Table 4 – Permitted geometry types per column	12

LIST OF FIGURES

Figure 1 – IndoorGML 2.0 core module structure	16
--	----

LIST OF NORMATIVE STATEMENTS

REQUIREMENTS CLASS 1	20
REQUIREMENTS CLASS 2	34
REQUIREMENT 1: REQUIREMENT FOR SQL ENCODING FEATURE TABLES	20
REQUIREMENT 2: INDOORFEATURES SCHEMA	21
REQUIREMENT 3: THEMATICLAYER SCHEMA	22
REQUIREMENT 4: PRIMALSPACELAYER SCHEMA	23
REQUIREMENT 5: CELLSPACE SCHEMA	25
REQUIREMENT 6: CELLBOUNDARY SCHEMA	26
REQUIREMENT 7: DUALSPACELAYER SCHEMA	27
REQUIREMENT 8: NODE SCHEMA	29
REQUIREMENT 9: EDGE SCHEMA	30
REQUIREMENT 10: INTERLAYERCONNECTION SCHEMA	32
REQUIREMENT 11: NAVIGABLESPACE SCHEMA	35
REQUIREMENT 12: GENERALSPACE SCHEMA	35
REQUIREMENT 13: TRANSFERSPACE SCHEMA	36
REQUIREMENT 14: OBJECTSPACE SCHEMA	37
REQUIREMENT 15: NAVIGABLEBOUNDARY SCHEMA	38
REQUIREMENT 16: ROUTE SCHEMA	39
CONFORMANCE CLASS A.1	41
CONFORMANCE CLASS A.2	45



ABSTRACT

The IndoorGML 2.0 Part 2c – SQL Encoding Standard presents the implementation-dependent Structured Query Language (SQL) encoding of the concepts defined by the IndoorGML 2.0 Part 1 – Conceptual Model Standard (OGC 22-045r5). The SQL Encoding is compliant with SQL as specified by ISO/IEC 9075 and is implemented using the object-relational database PostgreSQL as the reference database management system. The SQL encoding of IndoorGML 2.0 defines a concrete implementation schema for representing indoor spatial information, including primal space, dual space, thematic layers, interlayer connections, and navigation-related indoor features. This encoding can be used to store and maintain 2D and/or 3D indoor space and indoor navigation network models in object-relational databases and can be accessed via an Application Programming Interface (API) or via web services.

This Part 2c of the Standard defines how the concepts developed in Part 1 are realized using SQL table definitions. The SQL encoding represents a mapping of the Conceptual Model derived from the UML model using Enterprise Architecture tooling, with subsequent corrections to identifier types, relationships, and missing properties. Geometry values are stored in PostGIS geometry columns. Geometry DataTypes from the conceptual model are flattened onto owning feature tables using path-style column names (for example, cellSpaceGeom_Geometry2D), and non-geometric reference DataTypes are encoded as PostgreSQL composite types. Table 1 maps requirements classes from the IndoorGML 2.0 Conceptual Model into the implementation details documented in this standard.

Table 1

CONCEPTUAL MODEL	SECTION	SQL SCHEMA
IndoorGML Core	Clause 7	Annex B.1
Navigation Extension	Clause 8	Annex B.2



KEYWORDS

The following are keywords to be used by search engines and document catalogues.

OGCDoc, OGC documents, indoor, navigation, IndoorGML, SQL, PostgreSQL, PostGIS



PREFACE

In order to achieve consensus on the modeling indoor spaces and their neighborhood relationships to support indoor location-based services, a UML Conceptual Model, IndoorGML 2.0 Part 1, was approved as an OGC Standard (OGC 22-045r5) in June, 2025. This model covers geometric and semantic properties of indoor spaces relevant for indoor navigation. This IndoorGML 2.0 Part 2c: SQL Encoding Standard defines how those concepts should be realized using a relational database schema. The SQL encoding of IndoorGML 2.0 is intended for software developers, data providers, standards implementers, and service providers who need a persistent, queryable, and interoperable representation of IndoorGML datasets in SQL databases.



SECURITY CONSIDERATIONS

No security considerations have been made for this Standard.



SUBMITTING ORGANIZATIONS

The following organizations submitted this Document to the Open Geospatial Consortium (OGC):

- University of New South Wales
- SpatialAlgebra Technology Limited
- Pusan National University
- CityGeometrix Pty Ltd
- National Institute of Advanced Industrial Science and Technology



SUBMITTERS

All questions regarding this submission should be directed to the editor or the submitters:

NAME	AFFILIATION	EMAIL	OGC MEMBER
Sisi Zlatanova	University of New South Wales	s.zlatanova@unsw.edu.au	Yes
Zhiyong Wang	SpatialAlgebra Technology Limited	zhiyongwang@spatialalgebra.tech	Yes
Taehoon Kim	National Institute of Advanced Industrial Science and Technology	kim.taehoon@aist.go.jp	Yes
Abdoulaye Diakite	CityGeometrix Pty Ltd	abdou@citygeometrix.com	Yes
Ki-Joune Li	Pusan National University	lik@pnu.edu	Yes



1

SCOPE

The OGC IndoorGML 2.0 Part 2c – SQL Encoding Standard defines a SQL encoding for IndoorGML 2.0 Part 1 – Conceptual Model.

This Standard specifies:

- SQL table definitions for IndoorGML 2.0 core module;
- SQL table definitions for IndoorGML 2.0 navigation module;
- geometry storage rules for spatially enabled SQL database platforms, using PostgreSQL with PostGIS as the reference implementation; and
- a statement to which SQL encoding conformance classes an IndoorGML 2.0 implementation conforms to.

The SQL encoding of IndoorGML 2.0 is an implementation encoding of IndoorGML 2.0 Part 1. Therefore, database instances of the SQL encoding shall be interpreted according to the semantics defined in the IndoorGML 2.0 conceptual model.



2

CONFORMANCE

This standard defines an implementation specification which specifies how the IndoorGML 2.0 Conceptual Model should be implemented using Structured Query Language (SQL). The Standardization Target for this standard is:

- Implementations of the IndoorGML 2.0 Conceptual Model using SQL encodings.

Conformance with this standard shall be checked using all the relevant tests specified in Annex A (normative) of this document. The framework, concepts, and methodology for testing, and the criteria to be achieved to claim conformance are specified in the OGC Compliance Testing Policies and Procedures and the OGC Compliance Testing web site.

In order to conform to this OGC® interface standard, a software implementation shall choose to implement:

- Any one of the conformance levels specified in Annex A (normative).

All requirements-classes and conformance-classes described in this document are owned by the standard(s) identified.

Table 2

CONFORMANCE CLASS	URI
IndoorGML Core	http://www.opengis.net/spec/indoorgml-2/2.0/sql/conf/core
Navigation Extension	http://www.opengis.net/spec/indoorgml-2/2.0/sql/conf/navi



3

NORMATIVE REFERENCES

The following documents are referred to in the text in such a way that some or all of their content constitutes requirements of this document. For dated references, only the edition cited applies. For undated references, the latest edition of the referenced document (including any amendments) applies.

Sisi Zlatanova, Abdoulaye Diakite, Taehoon Kim, Ki-Joune Li: OGC 22-045r5, *OGC IndoorGML 2.0 Part 1 – Conceptual Model*. Open Geospatial Consortium (2025). <http://www.opengis.net/doc/IS/indoorgml/2.0>.

ISO: ISO 19101-1, *Geographic information – Reference model – Part 1: Fundamentals*. International Organization for Standardization, Geneva <https://www.iso.org/standard/59164.html>.

ISO: ISO 19103, *Geographic information – Conceptual schema language*. International Organization for Standardization, Geneva <https://www.iso.org/standard/83454.html>.

ISO: ISO 19107, *Geographic information – Spatial schema*. International Organization for Standardization, Geneva <https://www.iso.org/standard/66175.html>.

ISO/IEC: ISO/IEC 9075, *Information technology – Database languages – SQL*. International Organization for Standardization, International Electrotechnical Commission, Geneva <https://www.iso.org/standard/16663.html>.

PostGIS Spatial and Geographic Objects for PostgreSQL, <https://postgis.net/docs/>



4

TERMS AND DEFINITIONS

This document uses the terms defined in OGC Policy Directive 49, which is based on the ISO/IEC Directives, Part 2, Rules for the structure and drafting of International Standards. In particular, the word “shall” (not “must”) is the verb form used to indicate a requirement to be strictly followed to conform to this document and OGC documents do not use the equivalent phrases in the ISO/IEC Directives, Part 2.

This document also uses terms defined in the OGC Standard for Modular specifications (OGC 08-131r3), also known as the ‘ModSpec’. The definitions of terms such as standard, specification, requirement, and conformance test are provided in the ModSpec.

For the purposes of this document, the following additional terms and definitions apply.

4.1. conceptual model

model that defines concepts of a universe of discourse

[SOURCE: ISO 19101-1]

4.2. conceptual schema

formal description of a conceptual model

[SOURCE: ISO 19101-1]

4.3. data type

set of values that lack identity

Note 1 to entry: A data type is a class with no operations whose primary purpose is to hold information. In the IndoorGML 2.0 Conceptual Model, data types are denoted by the stereotype «DataType».

Note 2 to entry: In this SQL encoding, non-geometric UML DataTypes are encoded as PostgreSQL composite types, and geometry DataTypes are flattened onto owning feature tables as PostGIS geometry columns (see Clause 5).

[SOURCE: ISO 19103]

4.4. feature

abstraction of real world phenomena

Note 1 to entry: Features are objects that have an identity that allows for distinguishing features from each other. Features can have spatial and non-spatial properties that describe the features in more detail. Spatial properties are properties that have a geometry from ISO 19107 as data type.

Note 2 to entry: Features are denoted by the stereotype «FeatureType» in the IndoorGML 2.0 Conceptual Model.

[SOURCE: ISO 19101-1]

4.5. relational table

persistent data structure in a relational database that stores records of a single feature type or association

Note 1 to entry: In the SQL encoding of IndoorGML 2.0, each IndoorGML 2.0 feature class is mapped to one or more relational tables.



5

CONVENTIONS

5.1. Symbols (and abbreviated terms)

The following symbols and abbreviated terms are used in this standard. Symbols and abbreviated terms defined in IndoorGML 2.0 Part 1 also apply to this document.

Table 3 — Symbols and abbreviated terms

ABBREVIATION	WORD OR PHRASE
DDL	Data Definition Language
EA	Enterprise Architecture
JSON	JavaScript Object Notation
PostgreSQL	Open-source relational database management system
PostGIS	Spatial database extension for PostgreSQL
SQL	Structured Query Language

5.2. Identifiers

The normative provisions in this standard are denoted by the URI

<http://www.opengis.net/spec/indoorgml-2/2.0/sql>

All requirements and conformance tests that appear in this document are denoted by partial URIs which are relative to this base.

5.3. Use of SQL terminology

The following terms are used as specified in SQL. In particular:

- “table” refers to a named relation in the database schema;
- “column” refers to an attribute of a table; and
- “row” refers to a single record in a table.

Feature identifiers are stored as string values in varchar columns rather than integer auto-increment keys, in order to preserve cross-references between IndoorGML objects.

5.4. Geometry storage

UML geometry DataTypes are flattened onto the owning feature table as PostGIS geometry columns (PostGIS) whose names follow the UML property path. The reference DDL and examples in this document use PostgreSQL with PostGIS for geometry storage.

- `CellSpace.cellSpaceGeom_Geometry2D` and `CellSpace.cellSpaceGeom_Geometry3D` (corresponding to `CellSpaceGeometryType`);
- `CellBoundary.cellBoundaryGeom_Geometry1D` and `CellBoundary.cellBoundaryGeom_Geometry2D` (corresponding to `CellBoundaryGeometryType`);
- `Node.geometry` and `Edge.geometry` (corresponding to the geometry property of `Node` and `Edge` in the conceptual model).

Although a PostGIS geometry column can hold any geometry type, each column of this encoding is restricted to the geometry types that correspond to its UML geometry class. The permitted values of the PostGIS `GeometryType` function are as follows.

Table 4 — Permitted geometry types per column

COLUMN	UML GEOMETRY CLASS	PERMITTED <code>GeometryType</code> VALUES
<code>CellSpace.cellSpaceGeom_Geometry2D</code>	<code>GM_Surface</code>	POLYGON, MULTIPOLYGON
<code>CellSpace.cellSpaceGeom_Geometry3D</code>	<code>GM_Solid</code>	POLYHEDRALSURFACE, TIN
<code>CellBoundary.cellBoundaryGeom_Geometry1D</code>	<code>GM_Curve</code>	LINESTRING, MULTILINESTRING
<code>CellBoundary.cellBoundaryGeom_Geometry2D</code>	<code>GM_Surface</code>	POLYGON, MULTIPOLYGON
<code>Node.geometry</code>	<code>GM_Point</code>	POINT
<code>Edge.geometry</code>	<code>GM_Curve</code>	LINESTRING, MULTILINESTRING

These restrictions are enforced by table CHECK constraints in the DDL of Annex B and are stated normatively in the requirements of Clause 7.

NOTE 1: The Geometry1D, Geometry2D, and Geometry3D column-name suffixes denote the UML geometry class of the property, not the coordinate dimension of the stored value. The PostGIS GeometryType function does not distinguish coordinate dimension; for example, a POLYGON Z value has the geometry type POLYGON. Implementations that require planar geometry only MAY additionally constrain a column with ST_NDims(...) = 2.

NOTE 2: Implementations on other SQL platforms (for example, Oracle Spatial) MAY conform by preserving the same logical table structure, keys, associations, and cardinalities, and by mapping geometry columns and PostgreSQL-specific constructs (ENUM, composite types, jsonb) to equivalent platform-native types (for example, SDO_GEOMETRY).

NOTE 3: IndoorGML GM_Solid (UML CellSpaceGeometryType.Geometry3D / column cellSpaceGeom_Geometry3D) has no native solid primitive in PostGIS. It is therefore stored in a bare PostGIS geometry column as a closed boundary shell, typically POLYHEDRALSURFACE Z or TIN Z. A table CHECK constraint on CellSpace restricts non-null cellSpaceGeom_Geometry3D values to those geometry types. Volume and solid-topology operations (for example, ST_Volume, ST_MakeSolid, ST_IsSolid) require the optional PostGIS SFCGAL extension (postgis_sfcgal). Other SQL platforms MAY map the same logical column to a native solid type where available (for example, Oracle SDO_GEOMETRY solid).

5.5. Complex DataType storage

Non-geometric UML DataTypes that are not feature classes are encoded as PostgreSQL composite types (CREATE TYPE ... AS (...)) rather than as standalone tables. In particular, ExternalObjectReferenceType and ExternalReferenceType are composite types, and CellSpace / CellBoundary store an optional externalReference column of type ExternalReferenceType.

5.6. Code list storage

IndoorGML code list types (for example, ThemeLayerValue, TopoExpressionValue, LocomotionAccessType) are represented as PostgreSQL ENUM types whose labels match the UML type names and enumeration literals defined in the IndoorGML 2.0 conceptual model.

5.7. Graph and route associations

Graph adjacency in the dual space layer is encoded as follows:

- `Node.connects` stores the identifiers of connected Edge rows as a jsonb array;
- `Edge.connects_node1` and `Edge.connects_node2` store the two connected Node identifiers as foreign-key columns referencing `Node.NodeID`.

A Route represents an ordered navigation path through the dual-space graph. The ordered `routeNode` and `routeEdge` members are stored as jsonb arrays on the Route table rather than in separate junction tables. This encoding preserves the sequence of nodes and edges in a single row and simplifies maintenance of route order during import, update, and export. Array position expresses traversal order; the first element is the first node or edge in the route, the second element is the next, and so on. At least one of `routeNode` or `routeEdge` must be present for a valid Route row.

Example:

```
-- routeNode and routeEdge preserve traversal order in array position
INSERT INTO "Route" ("RouteID", "routeNode", "routeEdge")
VALUES (
    'R1',
    '['N1', 'N2', 'N3'] '::jsonb,
    '['E1', 'E2'] '::jsonb
);

-- Edge endpoints reference Node rows via foreign keys
INSERT INTO "Edge" ("EdgeID", "DualSpaceLayerID", "weight", connects_node1,
connects_node2)
VALUES ('E1', 'DS-1', 1.0, 'N1', 'N2');
```

Listing 1

5.8. Table and column naming

SQL table and column names use identifiers enclosed in double quotes. Table names use PascalCase aligned with UML type names (for example, `IndoorFeatures`, `ThematicLayer`, `CellSpace`, `NavigableSpace`, `ObjectSpace`).



6

OVERVIEW OF THE SQL ENCODING

The SQL encoding of IndoorGML 2.0 implements the conceptual classes of IndoorGML 2.0 Part 1 as relational database tables. The conceptual model remains normative for semantics, while this document defines the SQL representation.

6.1. Design Principle

The SQL encoding is based on the following encoding principles.

6.1.1. Conceptual model preservation

The SQL encoding of the IndoorGML core module preserves the IndoorGML 2.0 core conceptual model structure as close as possible, as shown in Figure 1.

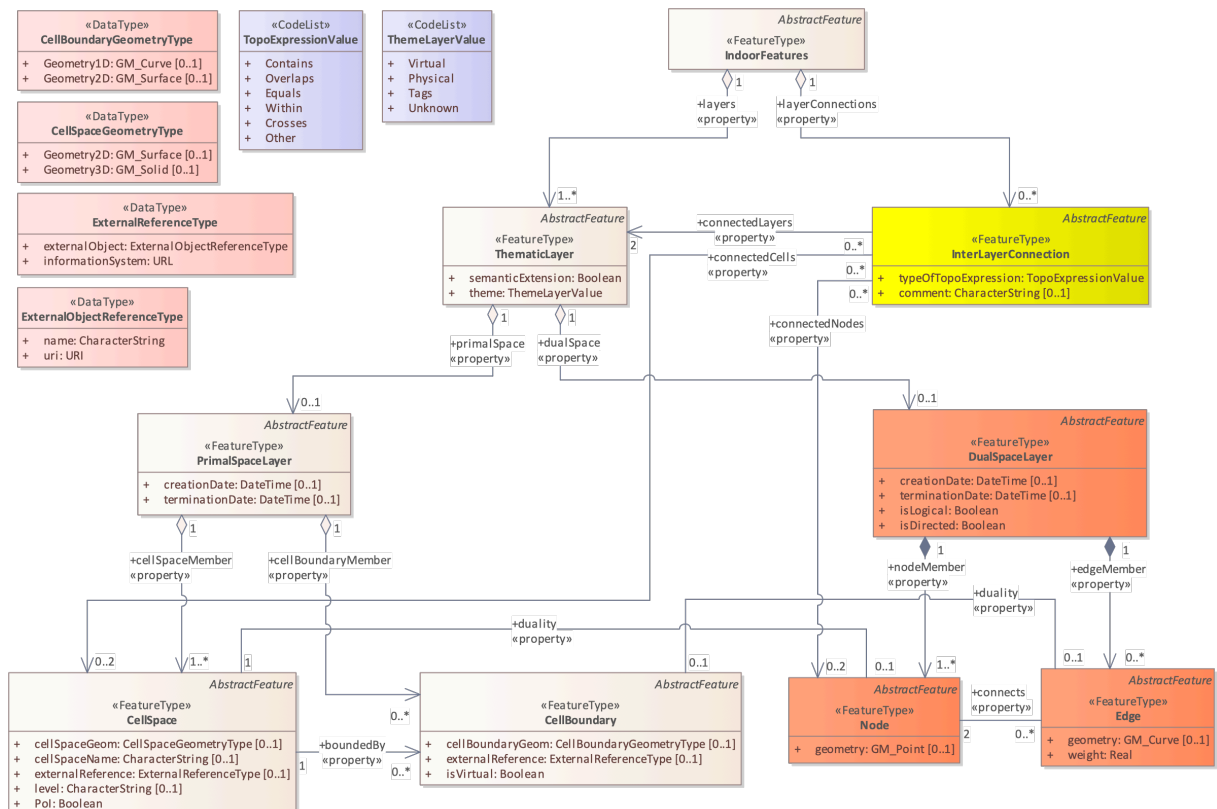


Figure 1 – IndoorGML 2.0 core module structure

Each feature class is mapped to one or more relational tables, and associations are represented using foreign key columns or junction tables. For example, the IndoorFeatures table is related to ThematicLayer rows through the IndoorFeaturesID foreign key, and

InterLayerConnection rows are linked to ThematicLayer, Node, and CellSpace rows through the InterLayerConnection_ThematicLayer, InterLayerConnection_Node, and InterLayerConnection_CellSpace junction tables.

The SQL encoding of the IndoorGML navigation module preserves the IndoorGML 2.0 navigation conceptual model structure as close as possible, as shown in Figure 2.

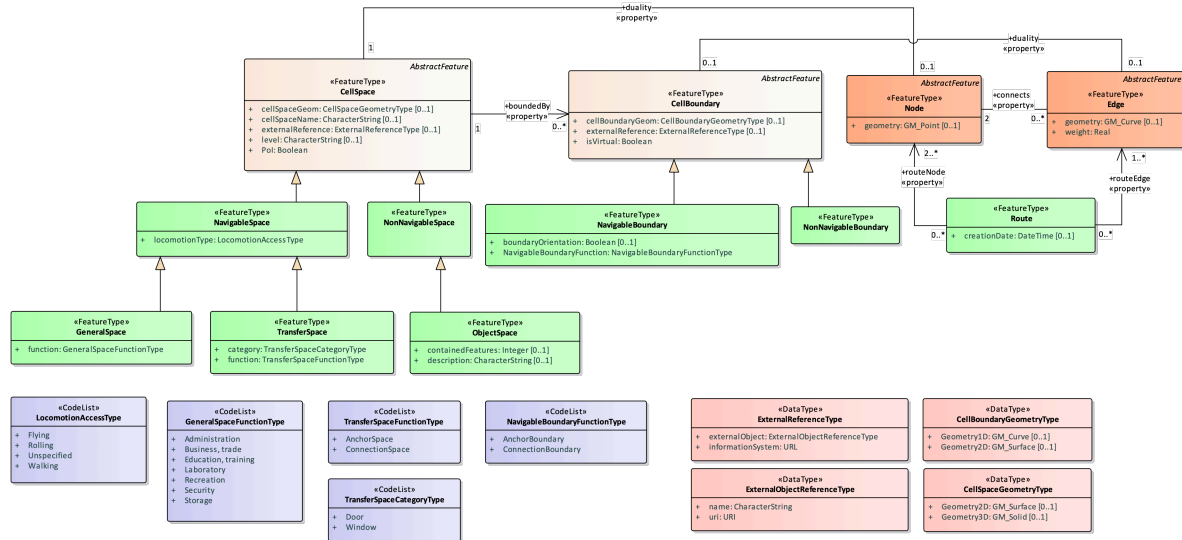


Figure 2 – IndoorGML 2.0 navigation module structure

Navigation feature classes extend core tables using table inheritance patterns implemented through foreign key references to CellSpace and CellBoundary.

6.1.2. UML-to-SQL generation workflow

The SQL encoding schema is derived from the IndoorGML 2.0 UML model and is designed on the basis of PostgreSQL with PostGIS as the reference database platform. SQL CREATE TABLE statements are first generated from the UML model using Enterprise Architecture tooling, and the resulting DDL is then corrected to align with the conceptual model. Typical corrections include:

- converting identifier properties from integer columns to string (varchar) columns;
- replacing EA code list tables with PostgreSQL ENUM types named after the UML code list types;
- flattening geometry DataType properties onto owning feature tables as path-named PostGIS columns (for example, cellSpaceGeom_Geometry2D / cellSpaceGeom_Geometry3D);
- replacing EA tables for ExternalReferenceType / ExternalObjectReferenceType with PostgreSQL composite types used as property columns;
- storing the Node.connects property as a jsonb array of connected Edge identifiers;

- storing the Edge endpoint properties as connects_node1 / connects_node2 foreign keys to Node;
- storing the routeNode and routeEdge properties as ordered jsonb arrays on Route to preserve traversal sequence;
- adding missing properties (for example, the layers property on IndoorFeatures);
- updating relationship cardinalities and junction tables.



7

SQL ENCODING OF THE CORE MODULE

REQUIREMENTS CLASS 1

IDENTIFIER	http://www.opengis.net/spec/indoorgml-2/2.0/sql/req/core
TARGET TYPE	Implementation Specification
CONFORMANCE CLASS	Conformance class A.1: http://www.opengis.net/spec/indoorgml-2/2.0/conf/sql/core
PREREQUISITE	http://www.opengis.net/spec/indoorgml/2.0/req
NORMATIVE STATEMENTS	Requirement 1: /req/core/indoorsql Requirement 2: /req/core/indoorfeature Requirement 3: /req/core/thematiclayer Requirement 4: /req/core/primalspacelayer Requirement 5: /req/core/cellspace Requirement 6: /req/core/cellboundary Requirement 7: /req/core/dualspacelayer Requirement 8: /req/core/node Requirement 9: /req/core/edge Requirement 10: /req/core/interlayerconnection

Each feature table of the SQL encoding represents one feature class of the IndoorGML 2.0 conceptual model.

Every feature table:

- **must** have a primary key column storing a string identifier that serves as a unique identifier for the feature and can be used to represent cross-references between objects in the SQL encoding.

REQUIREMENT 1: REQUIREMENT FOR SQL ENCODING FEATURE TABLES

IDENTIFIER	/req/core/indoorsql
INCLUDED IN	Requirements class 1: http://www.opengis.net/spec/indoorgml-2/2.0/sql/req/core
A	Each feature table of the SQL encoding SHALL contain a string primary key column for the feature identifier.

Core code list ENUM types used by the core module tables are specified in Annex C.

7.1. IndoorFeatures

The IndoorFeatures table stores instances of IndoorFeature, the container for the indoor dataset. It aggregates one or more thematic layers and may include inter-layer connections.

The IndoorFeatures table shall contain:

- IndoorFeaturesID as the primary key
- optionally, layers as a reference to associated thematic layers

NOTE:The layers column was added during schema correction because it was not included in the SQL generated from Enterprise Architecture tooling.

```
CREATE TABLE "IndoorFeatures"
(
    "IndoorFeaturesID" varchar(100) NOT NULL,
    "layers" varchar(100) NULL,
    CONSTRAINT "PK_IndoorFeatures" PRIMARY KEY ("IndoorFeaturesID")
);
```

Listing 2 – A SQL schema of the IndoorFeatures table

```
INSERT INTO "IndoorFeatures" ("IndoorFeaturesID", "layers")
VALUES ('IF-1', 'TL-1');

-- Additional thematic layers and inter-layer connections may be inserted here.
```

Listing 3 – An example INSERT statement for the IndoorFeatures table

Requirement 2: IndoorFeatures Schema	
IDENTIFIER	/req/core/indoorfeature
INCLUDED IN	Requirements class 1: http://www.opengis.net/spec/indoorqml-2/2.0/sql/req/core
A	An IndoorFeatures table SHALL be related to at least one ThematicLayer row through a foreign key.

7.2. ThematicLayer

The ThematicLayer table stores instances of ThematicLayer, which represents one contextual decomposition of indoor space.

The ThematicLayer table shall contain:

- ThematicLayerID as the primary key
- IndoorFeaturesID as a foreign key to IndoorFeatures
- semanticExtension indicates whether there is an extension module with additional semantic information (mandatory)
- theme of type ThemeLayerValue, with values including “Virtual”, “Physical”, “Tags”, and “Unknown” (mandatory)

```
CREATE TABLE "ThematicLayer"
(
    "semanticExtension" boolean NOT NULL,
    "theme" "ThemeLayerValue" NOT NULL,
    "ThematicLayerID" varchar(100) NOT NULL,
    "IndoorFeaturesID" varchar(100) NOT NULL,
    CONSTRAINT "PK_ThematicLayer" PRIMARY KEY ("ThematicLayerID"),
    CONSTRAINT "FK_ThematicLayer_layers"
        FOREIGN KEY ("IndoorFeaturesID") REFERENCES "IndoorFeatures"
        ("IndoorFeaturesID")
);

COMMENT ON TABLE "ThematicLayer"
    IS 'A layer of specific theme aggregating a primal and/or a dual space of a
    given environment.';

COMMENT ON COLUMN "ThematicLayer"."semanticExtension"
    IS 'Indicates whether semantic information is associated (true) or not
    (false) to the primal space of the thematic layer.';

COMMENT ON COLUMN "ThematicLayer"."theme"
    IS 'Determines the theme of the layer (e.g topographic, logical, etc.).';
```

Listing 4 – A SQL schema of the ThematicLayer table

REQUIREMENT 3: THEMATICLAYER SCHEMA

IDENTIFIER	/req/core/thematiclayer
INCLUDED IN	Requirements class 1: http://www.opengis.net/spec/indoorgml-2/2.0/sql/req/core
A	A ThematicLayer table SHALL declare whether there is an extension module with additional semantic information using the semanticExtension column.
B	A ThematicLayer table SHALL determine what type of model representation can be expected using the theme column.
C	The theme value SHALL be one of “Virtual”, “Physical”, “Tags”, and “Unknown”.

7.3. PrimalSpaceLayer

The PrimalSpaceLayer table stores instances of PrimalSpaceLayer, which contains spatial objects representing indoor subdivisions.

The PrimalSpaceLayer table shall contain:

- PrimalSpaceLayerID as the primary key
- ThematicLayerID as a foreign key to ThematicLayer (mandatory)
- optionally, creationDate
- optionally, terminationDate

```
CREATE TABLE "PrimalSpaceLayer"
(
    "creationDate" timestamp without time zone NULL,
    "terminationDate" timestamp without time zone NULL,
    "PrimalSpaceLayerID" varchar(100) NOT NULL,
    "ThematicLayerID" varchar(100) NOT NULL,
    CONSTRAINT "PK_PrimalSpaceLayer" PRIMARY KEY ("PrimalSpaceLayerID"),
    CONSTRAINT "FK_PrimalSpaceLayer_primalSpace"
        FOREIGN KEY ("ThematicLayerID") REFERENCES "ThematicLayer"
        ("ThematicLayerID")
);
```

Listing 5 – A SQL schema of the PrimalSpaceLayer table

Requirement 4: PrimalSpaceLayer Schema	
IDENTIFIER	/req/core/primalspacelayer
INCLUDED IN	Requirements class 1: http://www.opengis.net/spec/indoorqml-2/2.0/sql/req/core
A	A PrimalSpaceLayer table SHALL be related to one or more CellSpace rows.

7.4. CellSpace

The CellSpace table stores instances of CellSpace, which represents a unit of indoor space.

The CellSpace table shall contain:

- CellSpaceID as the primary key

- PrimalSpaceLayerID as a foreign key to PrimalSpaceLayer
- PoI is a flag that specifies whether this cell space is a point of interest (mandatory)
- duality referencing the NodeID of the dual node (mandatory)
- optionally, cellSpaceName
- optionally, level
- optionally, cellSpaceGeom_Geometry2D as a PostGIS geometry value (UML CellSpaceGeometryType.Geometry2D)
- optionally, cellSpaceGeom_Geometry3D as a PostGIS geometry value representing UML GM_Solid / CellSpaceGeometryType.Geometry3D as a closed boundary shell (POLYHEDRALSURFACE Z or TIN Z; see Clause 5)
- optionally, externalReference of composite type ExternalReferenceType

At most one of cellSpaceGeom_Geometry2D and cellSpaceGeom_Geometry3D is populated for a given row. When cellSpaceGeom_Geometry3D is present, a CHECK constraint requires GeometryType(...) to be POLYHEDRALSURFACE or TIN.

```
CREATE TABLE "CellSpace"
(
    "cellSpaceGeom_Geometry2D" geometry NULL,
    "cellSpaceGeom_Geometry3D" geometry NULL,
    "cellSpaceName" varchar(100) NULL,
    "externalReference" "ExternalReferenceType" NULL,
    "level" varchar(100) NULL,
    "PoI" boolean NOT NULL,
    "CellSpaceID" varchar(100) NOT NULL,
    "PrimalSpaceLayerID" varchar(100) NOT NULL,
    "duality" varchar(100) NOT NULL,
    CONSTRAINT "PK_CellSpace" PRIMARY KEY ("CellSpaceID"),
    CONSTRAINT "chk_CellSpace_geom_xor" CHECK (
        NOT ("cellSpaceGeom_Geometry2D" IS NOT NULL AND "cellSpaceGeom_
Geometry3D" IS NOT NULL)
    ),
    CONSTRAINT "chk_CellSpace_geom2d_surface" CHECK (
        "cellSpaceGeom_Geometry2D" IS NULL
        OR GeometryType("cellSpaceGeom_Geometry2D") IN ('POLYGON',
'MULTIPOLYGON')
    ),
    CONSTRAINT "chk_CellSpace_geom3d_solid" CHECK (
        "cellSpaceGeom_Geometry3D" IS NULL
        OR GeometryType("cellSpaceGeom_Geometry3D") IN ('POLYHEDRALSURFACE',
'TIN')
    ),
    CONSTRAINT "FK_CellSpace_cellSpaceMember"
        FOREIGN KEY ("PrimalSpaceLayerID") REFERENCES "PrimalSpaceLayer"
("PrimalSpaceLayerID")
);
```

Listing 6 – A SQL schema of the CellSpace table

```
SELECT
    "CellSpaceID"                AS id,
    'CellSpace'                  AS "featureType",
```

```

        "PoI"                AS poi,
        "cellSpaceGeom_Geometry2D" AS "geometry2D",
        "cellSpaceGeom_Geometry3D" AS "geometry3D",
        "cellSpaceName"         AS name,
        "duality"               AS duality,
        "level"                 AS level,
        "externalReference"     AS "externalReference"
FROM "CellSpace"
ORDER BY "CellSpaceID";

```

Listing 7 — An example query returning CellSpace records

REQUIREMENT 5: CELLSPACE SCHEMA

IDENTIFIER	/req/core/cellspace
INCLUDED IN	Requirements class 1: http://www.opengis.net/spec/indoorgml-2/2.0/sql/req/core
A	A CellSpace table SHALL declare whether it is a point of interest space using the PoI column.
B	A CellSpace table SHALL contain a duality value referencing the NodeID of the corresponding Node.
C	A CellSpace table SHALL NOT contain a row in which both cellSpaceGeom_Geometry2D and cellSpaceGeom_Geometry3D are non-null.
D	A non-null cellSpaceGeom_Geometry2D value SHALL have the geometry type POLYGON or MULTIPOLYGON.
E	A non-null cellSpaceGeom_Geometry3D value SHALL have the geometry type POLYHEDRALSURFACE or TIN.

7.5. CellBoundary

The CellBoundary table stores instances of CellBoundary, a boundary of a CellSpace. CellBoundary rows reference CellSpace rows through the boundedBy foreign key column. Optionally, a CellBoundary may be the dual of an Edge; this association is encoded on the Edge table via the optional Edge.duality foreign key referencing CellBoundaryID.

The CellBoundary table shall contain:

- CellBoundaryID as the primary key
- PrimalSpaceLayerID as a foreign key to PrimalSpaceLayer (mandatory)
- isVirtual indicating whether the boundary is virtual or physical (mandatory)
- optionally, boundedBy referencing the CellSpaceID of a bounded cell space

- optionally, cellBoundaryGeom_Geometry1D as a PostGIS geometry value (UML CellBoundaryGeometryType.Geometry1D, typically a curve)
- optionally, cellBoundaryGeom_Geometry2D as a PostGIS geometry value (UML CellBoundaryGeometryType.Geometry2D, typically a surface)
- optionally, externalReference of composite type ExternalReferenceType

At most one of cellBoundaryGeom_Geometry1D and cellBoundaryGeom_Geometry2D is populated for a given row.

```
CREATE TABLE "CellBoundary"
(
    "cellBoundaryGeom_Geometry1D" geometry NULL,
    "cellBoundaryGeom_Geometry2D" geometry NULL,
    "externalReference" "ExternalReferenceType" NULL,
    "isVirtual" boolean NOT NULL,
    "CellBoundaryID" varchar(100) NOT NULL,
    "boundedBy" varchar(100) NULL,
    "PrimalSpaceLayerID" varchar(100) NOT NULL,
    CONSTRAINT "PK_CellBoundary" PRIMARY KEY ("CellBoundaryID"),
    CONSTRAINT "chk_CellBoundary_geom_xor" CHECK (
        NOT ("cellBoundaryGeom_Geometry1D" IS NOT NULL AND "cellBoundaryGeom_
Geometry2D" IS NOT NULL)
    ),
    CONSTRAINT "chk_CellBoundary_geom1d_curve" CHECK (
        "cellBoundaryGeom_Geometry1D" IS NULL
        OR GeometryType("cellBoundaryGeom_Geometry1D") IN ('LINESTRING',
'MULTILINESTRING')
    ),
    CONSTRAINT "chk_CellBoundary_geom2d_surface" CHECK (
        "cellBoundaryGeom_Geometry2D" IS NULL
        OR GeometryType("cellBoundaryGeom_Geometry2D") IN ('POLYGON',
'MULTIPOLYGON')
    ),
    CONSTRAINT "FK_CellBoundary_boundedBy"
        FOREIGN KEY ("boundedBy") REFERENCES "CellSpace" ("CellSpaceID"),
    CONSTRAINT "FK_CellBoundary_cellBoundaryMember"
        FOREIGN KEY ("PrimalSpaceLayerID") REFERENCES "PrimalSpaceLayer"
("PrimalSpaceLayerID")
);
```

Listing 8 – A SQL schema of the CellBoundary table

REQUIREMENT 6: CELLBOUNDARY SCHEMA

IDENTIFIER /req/core/cellboundary

INCLUDED IN Requirements class 1: <http://www.opengis.net/spec/indoorgml-2/2.0/sql/req/core>

A A CellBoundary table SHALL specify whether it is virtual using the isVirtual column.

B A CellBoundary table SHALL NOT contain a row in which both cellBoundaryGeom_Geometry1D and cellBoundaryGeom_Geometry2D are non-null.

REQUIREMENT 6: CELLBOUNDARY SCHEMA

C	A non-null cellBoundaryGeom_Geometry1D value SHALL have the geometry type LINESTRING or MULTILINESTRING.
D	A non-null cellBoundaryGeom_Geometry2D value SHALL have the geometry type POLYGON or MULTIPOLYGON.

7.6. DualSpaceLayer

The DualSpaceLayer table stores instances of DualSpaceLayer, representing the graph structure corresponding to the primal space layer.

The DualSpaceLayer table shall contain:

- DualSpaceLayerID as the primary key
- ThematicLayerID as a foreign key to ThematicLayer (mandatory)
- isLogical indicating whether the graph is geometric or logical (mandatory)
- isDirected specifying whether the graph is directed (mandatory)
- optionally, creationDate
- optionally, terminationDate

```
CREATE TABLE "DualSpaceLayer"
(
    "creationDate" timestamp without time zone NULL,
    "terminationDate" timestamp without time zone NULL,
    "isLogical" boolean NOT NULL,
    "isDirected" boolean NOT NULL,
    "DualSpaceLayerID" varchar(100) NOT NULL,
    "ThematicLayerID" varchar(100) NOT NULL,
    CONSTRAINT "PK_DualSpaceLayer" PRIMARY KEY ("DualSpaceLayerID"),
    CONSTRAINT "FK_DualSpaceLayer_dualSpace"
        FOREIGN KEY ("ThematicLayerID") REFERENCES "ThematicLayer"
        ("ThematicLayerID")
);
```

Listing 9 — A SQL schema of the DualSpaceLayer table

REQUIREMENT 7: DUALSPACE LAYER SCHEMA

IDENTIFIER	/req/core/dualspace layer
INCLUDED IN	Requirements class 1: http://www.opengis.net/spec/indoorxml-2/2.0/sql/req/core

REQUIREMENT 7: DUALSPACE LAYER SCHEMA

A	A DualSpaceLayer table SHALL specify whether the provided graph is geometric or logical using the isLogical column.
B	A DualSpaceLayer table SHALL specify whether the provided graph is directed using the isDirected column.
C	A DualSpaceLayer table SHALL be related to one or more Node rows.

7.7. Node

The Node table stores instances of Node, a state or graph vertex in the dual space layer.

The Node table shall contain:

- NodeID as the primary key
- DualSpaceLayerID as a foreign key to DualSpaceLayer (mandatory)
- duality referencing the CellSpaceID of the corresponding cell space (mandatory)
- optionally, geometry as a PostGIS geometry value (typically Point)
- optionally, connects as a jsonb array of connected EdgeID values

```
CREATE TABLE "Node"
(
    "geometry" geometry NULL,
    "NodeID" varchar(100) NOT NULL,
    "duality" varchar(100) NOT NULL,
    "DualSpaceLayerID" varchar(100) NOT NULL,
    connects jsonb NULL,
    CONSTRAINT "PK_Node" PRIMARY KEY ("NodeID"),
    CONSTRAINT "chk_Node_geometry_point" CHECK (
        "geometry" IS NULL
        OR GeometryType("geometry") = 'POINT'
    ),
    CONSTRAINT "FK_Node_duality"
        FOREIGN KEY ("duality") REFERENCES "CellSpace" ("CellSpaceID"),
    CONSTRAINT "FK_Node_DualSpaceLayer"
        FOREIGN KEY ("DualSpaceLayerID") REFERENCES "DualSpaceLayer"
        ("DualSpaceLayerID")
);

ALTER TABLE "CellSpace" ADD CONSTRAINT "FK_CellSpace_duality"
    FOREIGN KEY ("duality") REFERENCES "Node" ("NodeID");
```

Listing 10 – A SQL schema of the Node table

```
SELECT
    n."NodeID" AS id,
    'Node' AS "featureType",
```

```

n."geometry" AS geometry,
n."duality" AS duality,
n.connects AS connects
FROM "Node" n
ORDER BY n."NodeID";

```

Listing 11 — An example query returning Node records

REQUIREMENT 8: NODE SCHEMA

IDENTIFIER	/req/core/node
INCLUDED IN	Requirements class 1: http://www.opengis.net/spec/indoorgml-2/2.0/sql/req/core
A	A Node table SHALL contain a duality value referencing the CellSpaceID of the corresponding CellSpace.
B	A Node table SHALL store connected edge identifiers in the connects column.
C	A non-null Node geometry value SHALL have the geometry type POINT.

7.8. Edge

The Edge table stores instances of Edge, representing a relationship between two nodes.

The Edge table shall contain:

- EdgeID as the primary key
- DualSpaceLayerID as a foreign key to DualSpaceLayer (mandatory)
- weight for use in graph-based applications (mandatory)
- connects_node1 and connects_node2 as foreign keys to the two connected Node rows (mandatory)
- optionally, duality referencing the CellBoundaryID of the corresponding cell boundary
- optionally, geometry as a PostGIS geometry value (typically LineString)

NOTE: If no list of connected edges is presented on Node (that is, Node.connects is absent or empty), the Edge table SHALL provide the connect properties connects_node1 and connects_node2 so that the dual-space graph adjacency can be determined from the Edge rows.

```

CREATE TABLE "Edge"
(
    "geometry" geometry NULL,
    "weight" real NOT NULL,
    "EdgeID" varchar(100) NOT NULL,

```

```

"duality" varchar(100) NULL,
"DualSpaceLayerID" varchar(100) NOT NULL,
connects_node1 varchar(100) NOT NULL,
connects_node2 varchar(100) NOT NULL,
CONSTRAINT "PK_Edge" PRIMARY KEY ("EdgeID"),
CONSTRAINT "chk_Edge_geometry_curve" CHECK (
    "geometry" IS NULL
    OR GeometryType("geometry") IN ('LINESTRING', 'MULTILINESTRING')
),
CONSTRAINT "FK_Edge_duality"
    FOREIGN KEY ("duality") REFERENCES "CellBoundary" ("CellBoundaryID"),
CONSTRAINT "FK_Edge_DualSpaceLayer"
    FOREIGN KEY ("DualSpaceLayerID") REFERENCES "DualSpaceLayer"
("DualSpaceLayerID"),
CONSTRAINT "FK_Edge_connects_node1"
    FOREIGN KEY (connects_node1) REFERENCES "Node" ("NodeID"),
CONSTRAINT "FK_Edge_connects_node2"
    FOREIGN KEY (connects_node2) REFERENCES "Node" ("NodeID")
);

```

Listing 12 — A SQL schema of the Edge table

REQUIREMENT 9: EDGE SCHEMA

IDENTIFIER /req/core/edge

INCLUDED IN Requirements class 1: <http://www.opengis.net/spec/indoorgml-2/2.0/sql/req/core>

A An Edge table SHALL reference connected Nodes using the connects_node1 and connects_node2 columns.

B An Edge table SHALL contain a weight column.

C A non-null Edge geometry value SHALL have the geometry type LINESTRING or MULTILINESTRING.

7.9. InterLayerConnection

The InterLayerConnection table stores instances of InterLayerConnection, representing a relationship between objects in different thematic layers.

The InterLayerConnection table shall contain:

- InterLayerConnectionID as the primary key
- IndoorFeaturesID as a foreign key to IndoorFeatures
- typeOfTopoExpression of type TopoExpressionValue, with values including “Contains”, “Overlaps”, “Equals”, “Within”, “Crosses”, and “Other” (mandatory)

- optionally, comment

Associated junction tables `InterLayerConnection_ThematicLayer`, `InterLayerConnection_Node`, and `InterLayerConnection_CellSpace` store references to thematic layers, nodes, and cell spaces respectively. They are split into three tables so that each many-to-many association targets a single feature class with a typed foreign key, rather than mixing heterogeneous identifiers in one column or array.

```
CREATE TABLE "InterLayerConnection"
(
    "typeOfTopoExpression" "TopoExpressionValue" NOT NULL,
    "comment" text NULL,
    "InterLayerConnectionID" varchar(100) NOT NULL,
    "IndoorFeaturesID" varchar(100) NULL,
    CONSTRAINT "PK_InterLayerConnection" PRIMARY KEY ("InterLayerConnectionID"),
    CONSTRAINT "FK_InterLayerConnection_layerConnections"
        FOREIGN KEY ("IndoorFeaturesID") REFERENCES "IndoorFeatures"
        ("IndoorFeaturesID")
);

COMMENT ON TABLE "InterLayerConnection"
    IS 'Describes the connection between two thematic layers. ';

COMMENT ON COLUMN "InterLayerConnection"."typeOfTopoExpression"
    IS 'Describes the topological relationship between two layers (e.g.
    overlaps, contains, etc.).';
```

Listing 13 – A SQL schema of the `InterLayerConnection` table

```
CREATE TABLE "InterLayerConnection_CellSpace"
(
    "connectedCells" varchar(100) NULL,
    "InterLayerConnectionID" varchar(100) NULL,
    CONSTRAINT "FK_InterLayerConnection_CellSpace_connectedCells"
        FOREIGN KEY ("connectedCells") REFERENCES "CellSpace" ("CellSpaceID"),
    CONSTRAINT "FK_InterLayerConnection_CellSpace_InterLayerConnection"
        FOREIGN KEY ("InterLayerConnectionID") REFERENCES "InterLayerConnection"
        ("InterLayerConnectionID")
);
```

Listing 14 – A SQL schema of the `InterLayerConnection_CellSpace` junction table

```
CREATE TABLE "InterLayerConnection_Node"
(
    "connectedNodes" varchar(100) NULL,
    "InterLayerConnectionID" varchar(100) NULL,
    CONSTRAINT "FK_InterLayerConnection_Node_connectedNodes"
        FOREIGN KEY ("connectedNodes") REFERENCES "Node" ("NodeID"),
    CONSTRAINT "FK_InterLayerConnection_Node_InterLayerConnection"
        FOREIGN KEY ("InterLayerConnectionID") REFERENCES "InterLayerConnection"
        ("InterLayerConnectionID")
);
```

Listing 15 – A SQL schema of the `InterLayerConnection_Node` junction table

```
CREATE TABLE "InterLayerConnection_ThematicLayer"
(
    "connectedLayers" varchar(100) NULL,
    "InterLayerConnectionID" varchar(100) NULL,
    CONSTRAINT "FK_InterLayerConnection_ThematicLayer_connectedLayers"
```



```

        FOREIGN KEY ("connectedLayers") REFERENCES "ThematicLayer"
("ThematicLayerID"),
        CONSTRAINT "FK_InterLayerConnection_ThematicLayer_InterLayerConnection"
        FOREIGN KEY ("InterLayerConnectionID") REFERENCES "InterLayerConnection"
("InterLayerConnectionID")
);

```

Listing 16 – A SQL schema of the InterLayerConnection_ThematicLayer junction table

REQUIREMENT 10: INTERLAYERCONNECTION SCHEMA

IDENTIFIER	/req/core/interlayerconnection
INCLUDED IN	Requirements class 1: http://www.opengis.net/spec/indoorgml-2/2.0/sql/req/core
A	An InterLayerConnection table SHALL reference ThematicLayer rows through the InterLayerConnection_ThematicLayer junction table.

7.10. ExternalReferenceType

The UML DataTypes ExternalObjectReferenceType and ExternalReferenceType are encoded as PostgreSQL composite types rather than as tables. These are not feature classes of IndoorGML 2.0. CellSpace and CellBoundary optionally store an externalReference column of type ExternalReferenceType, which nests an externalObject of type ExternalObjectReferenceType together with an informationSystem URI.

```

/* UML DataType ExternalObjectReferenceType / ExternalReferenceType
   Encoded as PostgreSQL composite types (not tables).
   Apply before creating CellSpace / CellBoundary. */

```

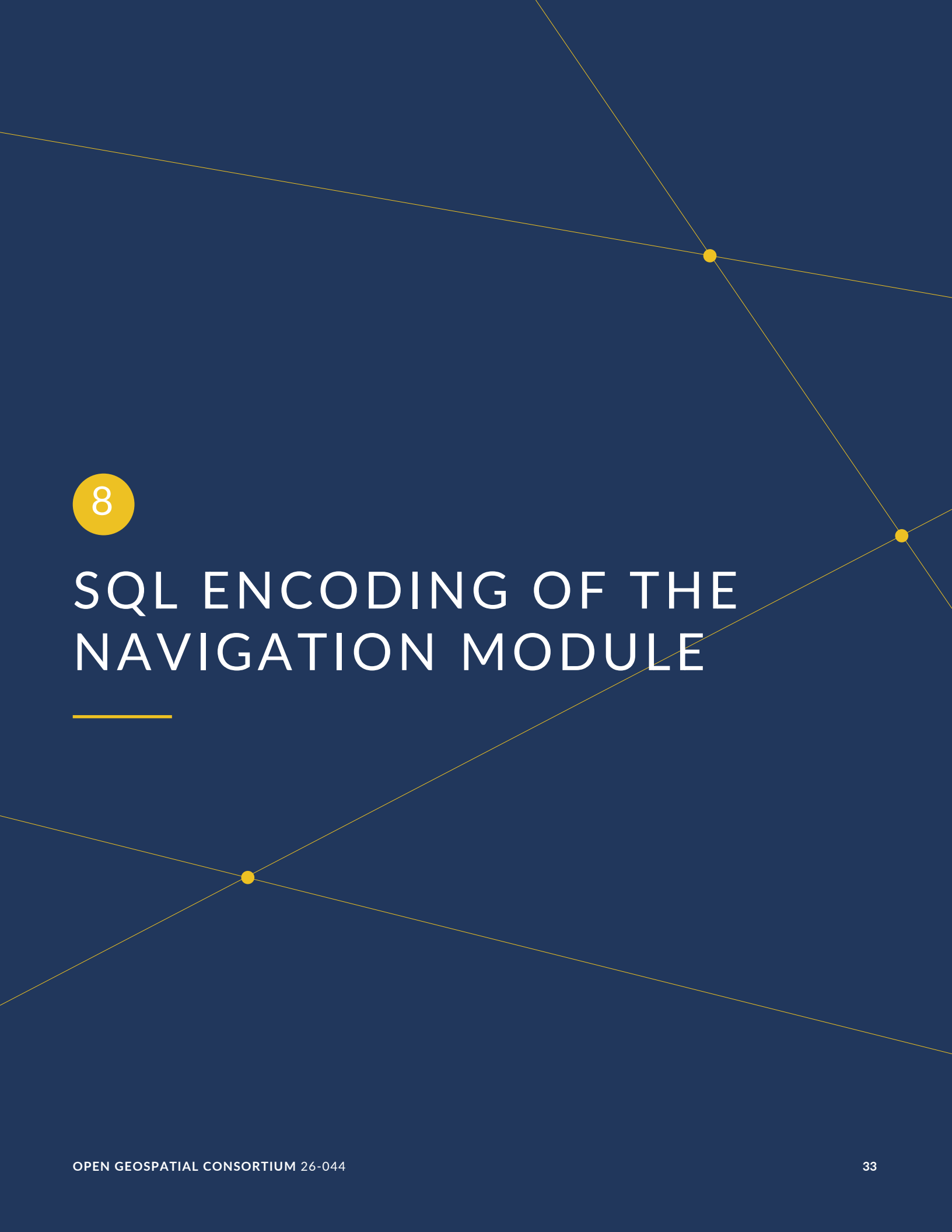
```

CREATE TYPE "ExternalObjectReferenceType" AS (
    "name" varchar(100),
    "uri" text
);

CREATE TYPE "ExternalReferenceType" AS (
    "externalObject" "ExternalObjectReferenceType",
    "informationSystem" text
);

```

Listing 17 – A SQL schema of the ExternalReference composite types



8

SQL ENCODING OF THE NAVIGATION MODULE

SQL ENCODING OF THE NAVIGATION MODULE

REQUIREMENTS CLASS 2	
IDENTIFIER	http://www.opengis.net/spec/indoorgml-2/2.0/sql/req/navi
TARGET TYPE	Implementation Specification
CONFORMANCE CLASS	Conformance class A.2: http://www.opengis.net/spec/indoorgml-2/2.0/conf/sql/navi
PREREQUISITE	http://www.opengis.net/spec/indoorgml/2.0/req
NORMATIVE STATEMENTS	Requirement 11: /req/navi/navigablespace Requirement 12: /req/navi/generalspace Requirement 13: /req/navi/transferspace Requirement 14: /req/navi/objectspace Requirement 15: /req/navi/navigableboundary Requirement 16: /req/navi/route

The navigation SQL encoding schema extends the core SQL encoding schema with tables for navigation-related feature classes.

NOTE: Navigation feature classes have not yet been validated with actual datasets in the preliminary implementation.

Navigation code list ENUM types used by the navigation module tables are specified in Annex D.

8.1. NavigableSpace

The NavigableSpace table extends CellSpace with navigation-specific attributes.

The NavigableSpace table shall contain:

- NavigableSpaceID as a primary key and foreign key to CellSpace
- locomotionType of type LocomotionAccessType, with values including "Walking", "Flying", "Rolling", and "Unspecified"

```
CREATE TABLE "NavigableSpace"
(
```

```

        "locomotionType" "LocomotionAccessType" NOT NULL,
        "NavigableSpaceID" varchar(100) NOT NULL,
        CONSTRAINT "PK_NavigableSpace" PRIMARY KEY ("NavigableSpaceID"),
        CONSTRAINT "FK_NavigableSpace_CellSpace"
            FOREIGN KEY ("NavigableSpaceID") REFERENCES "CellSpace" ("CellSpaceID")
    );

```

Listing 18 – A SQL schema of the NavigableSpace table

REQUIREMENT 11: NAVIGABLESPACE SCHEMA

IDENTIFIER	/req/navi/navigablespace
INCLUDED IN	Requirements class 2: http://www.opengis.net/spec/indoorgml-2/2.0/sql/req/navi
A	A NavigableSpace table SHALL reference a CellSpace row and SHALL contain a locomotionType column.

8.2. GeneralSpace

The GeneralSpace table extends NavigableSpace with a function attribute.

The GeneralSpace table shall contain:

- GeneralSpaceID as a primary key and foreign key to NavigableSpace
- function of type GeneralSpaceFunctionType, with values including “Administration”, “Business, trade”, “Education, training”, “Recreation”, “Laboratory”, “Storage”, and “Security”

```

CREATE TABLE "GeneralSpace"
(
    "function" "GeneralSpaceFunctionType" NOT NULL,
    "GeneralSpaceID" varchar(100) NOT NULL,
    CONSTRAINT "PK_GeneralSpace" PRIMARY KEY ("GeneralSpaceID"),
    CONSTRAINT "FK_GeneralSpace_NavigableSpace"
        FOREIGN KEY ("GeneralSpaceID") REFERENCES "NavigableSpace"
        ("NavigableSpaceID")
);

```

Listing 19 – A SQL schema of the GeneralSpace table

REQUIREMENT 12: GENERALSPEACE SCHEMA

IDENTIFIER	/req/navi/generalspace
INCLUDED IN	Requirements class 2: http://www.opengis.net/spec/indoorgml-2/2.0/sql/req/navi

REQUIREMENT 12: GENERALSPEACE SCHEMA

A	A GeneralSpace table SHALL contain a function column.
---	---

8.3. TransferSpace

The TransferSpace table extends NavigableSpace with transfer-specific attributes.

The TransferSpace table shall contain:

- TransferSpaceID as a primary key and foreign key to NavigableSpace
- category of type TransferSpaceCategoryType, with values “Door” and “Window”
- function of type TransferSpaceFunctionType, with values “AnchorSpace” and “ConnectionSpace”

```
CREATE TABLE "TransferSpace"
(
    "function" "TransferSpaceFunctionType" NOT NULL,
    "category" "TransferSpaceCategoryType" NOT NULL,
    "TransferSpaceID" varchar(100) NOT NULL,
    CONSTRAINT "PK_TransferSpace" PRIMARY KEY ("TransferSpaceID"),
    CONSTRAINT "FK_TransferSpace_NavigableSpace"
        FOREIGN KEY ("TransferSpaceID") REFERENCES "NavigableSpace"
        ("NavigableSpaceID")
);
```

Listing 20 — A SQL schema of the TransferSpace table

REQUIREMENT 13: TRANSFERSPACE SCHEMA

IDENTIFIER	/req/navi/transferspace
INCLUDED IN	Requirements class 2: http://www.opengis.net/spec/indoorgml-2/2.0/sql/req/navi
A	A TransferSpace table SHALL contain category and function columns.

8.4. NonNavigableSpace

The NonNavigableSpace table extends CellSpace for spaces that are not navigable.

```
CREATE TABLE "NonNavigableSpace"
(
    "NonNavigableSpaceID" varchar(100) NOT NULL,
    CONSTRAINT "PK_NonNavigableSpace" PRIMARY KEY ("NonNavigableSpaceID"),
    CONSTRAINT "FK_NonNavigableSpace_CellSpace"
        FOREIGN KEY ("NonNavigableSpaceID") REFERENCES "CellSpace"
        ("CellSpaceID")
);
```

Listing 21 – A SQL schema of the NonNavigableSpace table

8.5. ObjectSpace

The ObjectSpace table extends NonNavigableSpace with object-specific attributes.

The ObjectSpace table shall contain:

- ObjectSpaceID as a primary key and foreign key to NonNavigableSpace
- optionally, containedFeatures as an integer count of contained objects
- optionally, description

```
CREATE TABLE "ObjectSpace"
(
    "description" text NULL,
    "containedFeatures" integer NULL,
    "ObjectSpaceID" varchar(100) NOT NULL,
    CONSTRAINT "PK_ObjectSpace" PRIMARY KEY ("ObjectSpaceID"),
    CONSTRAINT "FK_ObjectSpace_NonNavigableSpace"
        FOREIGN KEY ("ObjectSpaceID") REFERENCES "NonNavigableSpace"
        ("NonNavigableSpaceID")
);
```

Listing 22 – A SQL schema of the ObjectSpace table

Requirement 14: ObjectSpace Schema	
IDENTIFIER	/req/navi/objectspace
INCLUDED IN	Requirements class 2: http://www.opengis.net/spec/indoorqml-2/2.0/sql/req/navi
A	An ObjectSpace table SHALL reference a NonNavigableSpace row.

8.6. NonNavigableBoundary

The NonNavigableBoundary table extends CellBoundary for boundaries that are not navigable.

```
CREATE TABLE "NonNavigableBoundary"
(
    "NonNavigableBoundaryID" varchar(100) NOT NULL,
    CONSTRAINT "PK_NonNavigableBoundary" PRIMARY KEY ("NonNavigableBoundaryID"),
    CONSTRAINT "FK_NonNavigableBoundary_CellBoundary"
        FOREIGN KEY ("NonNavigableBoundaryID") REFERENCES "CellBoundary"
        ("CellBoundaryID")
);
```

Listing 23 – A SQL schema of the NonNavigableBoundary table

8.7. NavigableBoundary

The NavigableBoundary table extends CellBoundary with navigation-specific boundary attributes.

The NavigableBoundary table shall contain:

- NavigableBoundaryID as a primary key and foreign key to CellBoundary
- optionally, boundaryOrientation
- NavigableBoundaryFunction of type NavigableBoundaryFunctionType, with values “AnchorBoundary” and “ConnectionBoundary”

```
CREATE TABLE "NavigableBoundary"
(
    "NavigableBoundaryFunction" "NavigableBoundaryFunctionType" NOT NULL,
    "boundaryOrientation" boolean NULL,
    "NavigableBoundaryID" varchar(100) NOT NULL,
    CONSTRAINT "PK_NavigableBoundary" PRIMARY KEY ("NavigableBoundaryID"),
    CONSTRAINT "FK_NavigableBoundary_CellBoundary"
        FOREIGN KEY ("NavigableBoundaryID") REFERENCES "CellBoundary"
        ("CellBoundaryID")
);
```

Listing 24 – A SQL schema of the NavigableBoundary table

Requirement 15: NavigableBoundary Schema	
IDENTIFIER	/req/navi/navigableboundary
INCLUDED IN	Requirements class 2: http://www.opengis.net/spec/indoorqml-2/2.0/sql/req/navi

REQUIREMENT 15: NAVIGABLEBOUNDARY SCHEMA

A	A NavigableBoundary table SHALL contain a NavigableBoundaryFunction column.
---	---

8.8. Route

The Route table represents a navigation route as an ordered sequence of nodes and edges.

In this SQL encoding, routeNode and routeEdge are stored as jsonb arrays on the Route row. This design keeps the full traversal order in one place and makes it easier to maintain, reorder, or replace the node and edge sequence of a route without managing separate ordered junction-table rows. The array index corresponds to position in the route: routeNode[0] is the first node, routeNode[1] is the second node, and routeEdge follows the same rule for edges. At least one of routeNode or routeEdge shall be present and non-empty to express a valid Route.

The Route table shall contain:

- RouteID as the primary key
- optionally, creationDate
- optionally, routeNode as a jsonb array of ordered NodeID values
- optionally, routeEdge as a jsonb array of ordered EdgeID values

```
CREATE TABLE "Route"
(
    "creationDate" timestamp without time zone NULL,
    "RouteID" varchar(100) NOT NULL,
    "routeNode" jsonb NULL,
    "routeEdge" jsonb NULL,
    CONSTRAINT "PK_Route" PRIMARY KEY ("RouteID")
);
```

Listing 25 – A SQL schema of the Route table

REQUIREMENT 16: ROUTE SCHEMA

IDENTIFIER	/req/navi/route
INCLUDED IN	Requirements class 2: http://www.opengis.net/spec/indoorqml-2/2.0/sql/req/navi
A	A Route table SHALL store ordered node identifiers in the routeNode column.
B	A Route table SHALL store ordered edge identifiers in the routeEdge column.



ANNEX A (NORMATIVE) CONFORMANCE CLASS ABSTRACT TEST SUITE



ANNEX A

(NORMATIVE)

CONFORMANCE CLASS ABSTRACT TEST SUITE

A.1. Conformance Class Core

CONFORMANCE CLASS A.1	
IDENTIFIER	<code>http://www.opengis.net/spec/indoorgml-2/2.0/conf/sql/core</code>
REQUIREMENTS CLASS	Requirements class 1: <code>http://www.opengis.net/spec/indoorgml-2/2.0/sql/req/core</code>
TARGET TYPE	Implementation Specification
CONFORMANCE TESTS	Abstract test A.1: <code>/conf/core/indoorsql</code> Abstract test A.2: <code>/conf/core/indoorfeature</code> Abstract test A.3: <code>/conf/core/thematiclayer</code> Abstract test A.4: <code>/conf/core/primalspacelayer</code> Abstract test A.5: <code>/conf/core/cellspace</code> Abstract test A.6: <code>/conf/core/cellboundary</code> Abstract test A.7: <code>/conf/core/dualspacelayer</code> Abstract test A.8: <code>/conf/core/node</code> Abstract test A.9: <code>/conf/core/edge</code> Abstract test A.10: <code>/conf/core/interlayerconnection</code>

ABSTRACT TEST A.1	
IDENTIFIER	<code>/conf/core/indoorsql</code>
REQUIREMENT	Requirement 1: <code>/req/core/indoorsql</code>
TEST PURPOSE	Verify that each feature table of the SQL encoding contains a string primary key column for the feature identifier.

ABSTRACT TEST A.1

TEST METHOD Inspect the database schema to verify the above requirement.

ABSTRACT TEST A.2

IDENTIFIER /conf/core/indoorfeature

REQUIREMENT Requirement 2: /req/core/indoorfeature

TEST PURPOSE Verify that an IndoorFeatures table is related to at least one ThematicLayer row through a foreign key.

TEST METHOD Inspect the database schema and instance data to verify the above requirement.

ABSTRACT TEST A.3

IDENTIFIER /conf/core/thematiclayer

REQUIREMENT Requirement 3: /req/core/thematiclayer

TEST PURPOSE For the following columns of the ThematicLayer table, verify that:

- The semanticExtension column declares whether there is an extension module with additional semantic information.
- The theme column determines what type of model representation can be expected.
- The theme value is one of "Virtual", "Physical", "Tags", and "Unknown".

TEST METHOD Inspect the database schema and instance data to verify the above requirement.

ABSTRACT TEST A.4

IDENTIFIER /conf/core/primalspacelayer

REQUIREMENT Requirement 4: /req/core/primalspacelayer

TEST PURPOSE Verify that a PrimalSpaceLayer table is related to one or more CellSpace rows.

TEST METHOD Inspect the database schema and instance data to verify the above requirement.

ABSTRACT TEST A.5

IDENTIFIER /conf/core/cellspace

REQUIREMENT Requirement 5: /req/core/cellspace

TEST PURPOSE

For the following columns of the CellSpace table, verify that:

- The PoI column declares whether it is a point of interest space.
- The duality column references the NodeID of the corresponding Node.
- No row has both cellSpaceGeom_Geometry2D and cellSpaceGeom_Geometry3D non-null.
- Every non-null cellSpaceGeom_Geometry2D value has the geometry type POLYGON or MULTIPOLYGON.
- Every non-null cellSpaceGeom_Geometry3D value has the geometry type POLYHEDRALSURFACE or TIN.

TEST METHOD Inspect the database schema and instance data to verify the above requirement.

ABSTRACT TEST A.6

IDENTIFIER /conf/core/cellboundary

REQUIREMENT Requirement 6: /req/core/cellboundary

TEST PURPOSE

For the following columns of the CellBoundary table, verify that:

- The isVirtual column specifies whether the boundary is virtual.
- No row has both cellBoundaryGeom_Geometry1D and cellBoundaryGeom_Geometry2D non-null.
- Every non-null cellBoundaryGeom_Geometry1D value has the geometry type LINESTRING or MULTILINESTRING.
- Every non-null cellBoundaryGeom_Geometry2D value has the geometry type POLYGON or MULTIPOLYGON.

TEST METHOD Inspect the database schema and instance data to verify the above requirement.

ABSTRACT TEST A.7

IDENTIFIER /conf/core/dualspacelayer

REQUIREMENT Requirement 7: /req/core/dualspacelayer

TEST PURPOSE

For the following columns and associations of the DualSpaceLayer table, verify that:

- The isLogical column specifies whether the provided graph is geometric or logical.

ABSTRACT TEST A.7

- The `isDirected` column specifies whether the provided graph is directed.
- The `DualSpaceLayer` table is related to one or more `Node` rows.

TEST METHOD Inspect the database schema and instance data to verify the above requirement.

ABSTRACT TEST A.8

IDENTIFIER `/conf/core/node`

REQUIREMENT Requirement 8: `/req/core/node`

TEST PURPOSE For the following columns of the `Node` table, verify that:

- The `duality` column references the `CellSpaceID` of the corresponding `CellSpace`.
- Connected edge identifiers are stored in the `connects` column.
- Every non-null geometry value has the geometry type `POINT`.

TEST METHOD Inspect the database schema and instance data to verify the above requirement.

ABSTRACT TEST A.9

IDENTIFIER `/conf/core/edge`

REQUIREMENT Requirement 9: `/req/core/edge`

TEST PURPOSE For the following columns of the `Edge` table, verify that:

- Connected `Node` rows are referenced using the `connects_node1` and `connects_node2` columns.
- A `weight` column is present.
- Every non-null geometry value has the geometry type `LINESTRING` or `MULTILINESTRING`.

TEST METHOD Inspect the database schema and instance data to verify the above requirement.

ABSTRACT TEST A.10

IDENTIFIER `/conf/core/interlayerconnection`

REQUIREMENT Requirement 10: `/req/core/interlayerconnection`

ABSTRACT TEST A.10

TEST PURPOSE Verify that an InterLayerConnection table references ThematicLayer rows through the InterLayerConnection_ThematicLayer junction table.

TEST METHOD Inspect the database schema and instance data to verify the above requirement.

A.2. Conformance Class Navigation

CONFORMANCE CLASS A.2

IDENTIFIER <http://www.opengis.net/spec/indoorgml-2/2.0/conf/sql/navi>

REQUIREMENTS CLASS Requirements class 2: <http://www.opengis.net/spec/indoorgml-2/2.0/sql/req/navi>

TARGET TYPE Implementation Specification

CONFORMANCE TESTS

- Abstract test A.11: /conf/navi/navigablespace
- Abstract test A.12: /conf/navi/generalspace
- Abstract test A.13: /conf/navi/transferspace
- Abstract test A.14: /conf/navi/objectspace
- Abstract test A.15: /conf/navi/navigableboundary
- Abstract test A.16: /conf/navi/route

ABSTRACT TEST A.11

IDENTIFIER /conf/navi/navigablespace

REQUIREMENT Requirement 11: /req/navi/navigablespace

TEST PURPOSE Verify that a NavigableSpace table references a CellSpace row and contains a locomotionType column.

TEST METHOD Inspect the database schema and instance data to verify the above requirement.

ABSTRACT TEST A.12

IDENTIFIER /conf/navi/generalspace

ABSTRACT TEST A.12

REQUIREMENT	Requirement 12: /req/navi/generalspace
TEST PURPOSE	Verify that a GeneralSpace table contains a function column.
TEST METHOD	Inspect the database schema and instance data to verify the above requirement.

ABSTRACT TEST A.13

IDENTIFIER	/conf/navi/transferspace
REQUIREMENT	Requirement 13: /req/navi/transferspace
TEST PURPOSE	Verify that a TransferSpace table contains category and function columns.
TEST METHOD	Inspect the database schema and instance data to verify the above requirement.

ABSTRACT TEST A.14

IDENTIFIER	/conf/navi/objectspace
REQUIREMENT	Requirement 14: /req/navi/objectspace
TEST PURPOSE	Verify that an ObjectSpace table references a NonNavigableSpace row.
TEST METHOD	Inspect the database schema and instance data to verify the above requirement.

ABSTRACT TEST A.15

IDENTIFIER	/conf/navi/navigableboundary
REQUIREMENT	Requirement 15: /req/navi/navigableboundary
TEST PURPOSE	Verify that a NavigableBoundary table contains a NavigableBoundaryFunction column.
TEST METHOD	Inspect the database schema and instance data to verify the above requirement.

ABSTRACT TEST A.16

IDENTIFIER /conf/navi/route

REQUIREMENT Requirement 16: /req/navi/route

TEST PURPOSE For the following columns of the Route table, verify that:

- Ordered node identifiers are stored in the routeNode column.
- Ordered edge identifiers are stored in the routeEdge column.

TEST METHOD Inspect the database schema and instance data to verify the above requirement.



ANNEX B (NORMATIVE) SCHEMAS

B

ANNEX B (NORMATIVE) SCHEMAS

NOTE: The DDL listings in this annex are expressed for the PostgreSQL/PostGIS reference platform. Implementations on other SQL platforms (for example, Oracle Spatial) MAY apply an equivalent mapping that preserves the logical table structure, keys, associations, and cardinalities defined by this Standard (see Clause 5).

B.1. SQL Schema of IndoorGML Core

```
/* ----- */
/* Generated by Enterprise Architect Version 14.1 */
/* Created On : 07-Jul-2025 11:10:23 */
/* DBMS : PostgreSQL */
/* ----- */

CREATE EXTENSION IF NOT EXISTS postgis
;
/* Drop Tables */

DROP TABLE IF EXISTS "CellBoundary" CASCADE
;

DROP TABLE IF EXISTS "CellBoundaryGeometryType" CASCADE
;

DROP TABLE IF EXISTS "CellSpace" CASCADE
;

DROP TABLE IF EXISTS "CellSpaceGeometryType" CASCADE
;

DROP TABLE IF EXISTS "DualSpaceLayer" CASCADE
;

DROP TABLE IF EXISTS "Edge" CASCADE
;

DROP TABLE IF EXISTS "IndoorFeatures" CASCADE
;

DROP TABLE IF EXISTS "InterLayerConnection" CASCADE
;

DROP TABLE IF EXISTS "InterLayerConnection_CellSpace" CASCADE
;
```

```

DROP TABLE IF EXISTS "InterLayerConnection_Node" CASCADE
;

DROP TABLE IF EXISTS "InterLayerConnection_ThematicLayer" CASCADE
;

DROP TABLE IF EXISTS "InterLayerConnection CellSpace" CASCADE
;

DROP TABLE IF EXISTS "InterLayerConnection Node" CASCADE
;

DROP TABLE IF EXISTS "InterLayerConnection ThematicLayer" CASCADE
;

DROP TABLE IF EXISTS "Node" CASCADE
;

DROP TABLE IF EXISTS "PrimalSpaceLayer" CASCADE
;

DROP TABLE IF EXISTS "ThematicLayer" CASCADE
;

/* Drop ENUM and composite Types */

DROP TYPE IF EXISTS "TopoExpressionValue" CASCADE
;

DROP TYPE IF EXISTS "ThemeLayerValue" CASCADE
;

DROP TYPE IF EXISTS "ExternalReferenceType" CASCADE
;

DROP TYPE IF EXISTS "ExternalObjectReferenceType" CASCADE
;

/* Create ENUM Types */

CREATE TYPE "ThemeLayerValue" AS ENUM (
    'Virtual',
    'Physical',
    'Tags',
    'Unknown'
)
;

CREATE TYPE "TopoExpressionValue" AS ENUM (
    'Contains',
    'Overlaps',
    'Equals',
    'Within',
    'Crosses',
    'Other'
)
;

/* Create composite Types (UML DataTypes for external references) */

CREATE TYPE "ExternalObjectReferenceType" AS (
    "name" varchar(100),

```

```

"uri" text
)
;

CREATE TYPE "ExternalReferenceType" AS (
    "externalObject" "ExternalObjectReferenceType",
    "informationSystem" text
)
;

/* Create Tables */

CREATE TABLE "CellBoundary"
(
    "cellBoundaryGeom_Geometry1D" geometry NULL,
    "cellBoundaryGeom_Geometry2D" geometry NULL,
    "externalReference" "ExternalReferenceType" NULL,
    "isVirtual" boolean NOT NULL,
    "CellBoundaryID" varchar(100) NOT NULL,
    "boundedBy" varchar(100) NULL,
    "PrimalSpaceLayerID" varchar(100) NOT NULL,
    CONSTRAINT "chk_CellBoundary_geom_xor" CHECK (
        NOT ("cellBoundaryGeom_Geometry1D" IS NOT NULL AND "cellBoundaryGeom_
        Geometry2D" IS NOT NULL)
    ),
    CONSTRAINT "chk_CellBoundary_geom1d_curve" CHECK (
        "cellBoundaryGeom_Geometry1D" IS NULL
        OR GeometryType("cellBoundaryGeom_Geometry1D") IN ('LINESTRING',
        'MULTILINESTRING')
    ),
    CONSTRAINT "chk_CellBoundary_geom2d_surface" CHECK (
        "cellBoundaryGeom_Geometry2D" IS NULL
        OR GeometryType("cellBoundaryGeom_Geometry2D") IN ('POLYGON', 'MULTIPOLYGON')
    )
)
;

CREATE TABLE "CellSpace"
(
    "cellSpaceGeom_Geometry2D" geometry NULL,
    "cellSpaceGeom_Geometry3D" geometry NULL,
    "cellSpaceName" varchar(100) NULL,
    "externalReference" "ExternalReferenceType" NULL,
    "level" varchar(100) NULL,
    "PoI" boolean NOT NULL,
    "CellSpaceID" varchar(100) NOT NULL,
    "PrimalSpaceLayerID" varchar(100) NOT NULL,
    "duality" varchar(100) NOT NULL,
    CONSTRAINT "chk_CellSpace_geom_xor" CHECK (
        NOT ("cellSpaceGeom_Geometry2D" IS NOT NULL AND "cellSpaceGeom_Geometry3D" IS
        NOT NULL)
    ),
    CONSTRAINT "chk_CellSpace_geom2d_surface" CHECK (
        "cellSpaceGeom_Geometry2D" IS NULL
        OR GeometryType("cellSpaceGeom_Geometry2D") IN ('POLYGON', 'MULTIPOLYGON')
    ),
    CONSTRAINT "chk_CellSpace_geom3d_solid" CHECK (
        "cellSpaceGeom_Geometry3D" IS NULL
        OR GeometryType("cellSpaceGeom_Geometry3D") IN ('POLYHEDRALSURFACE', 'TIN')
    )
)
;

```

```

CREATE TABLE "DualSpaceLayer"
(
    "creationDate" timestamp without time zone NULL,
    "terminationDate" timestamp without time zone NULL,
    "isLogical" boolean NOT NULL,
    "isDirected" boolean NOT NULL,
    "DualSpaceLayerID" varchar(100) NOT NULL,
    "ThematicLayerID" varchar(100) NOT NULL
)
;

CREATE TABLE "Edge"
(
    "geometry" geometry NULL,
    "weight" real NOT NULL,
    "EdgeID" varchar(100) NOT NULL,
    "duality" varchar(100) NULL,
    "DualSpaceLayerID" varchar(100) NOT NULL,
    connects_node1 varchar(100) NOT NULL,
    connects_node2 varchar(100) NOT NULL,
    CONSTRAINT "chk_Edge_geometry_curve" CHECK (
        "geometry" IS NULL
        OR GeometryType("geometry") IN ('LINESTRING', 'MULTILINESTRING')
    )
)
;

CREATE TABLE "IndoorFeatures"
(
    "IndoorFeaturesID" varchar(100) NOT NULL,
    "layers" varchar(100) NULL -- Reference to layers if needed, though usually
layers point back to IndoorFeatures
)
;

CREATE TABLE "InterLayerConnection"
(
    "typeOfTopoExpression" "TopoExpressionValue" NOT NULL,
    "comment" text NULL,
    "InterLayerConnectionID" varchar(100) NOT NULL,
    "IndoorFeaturesID" varchar(100) NULL
)
;

CREATE TABLE "InterLayerConnection_CellSpace"
(
    "connectedCells" varchar(100) NULL,
    "InterLayerConnectionID" varchar(100) NULL
)
;

CREATE TABLE "InterLayerConnection_Node"
(
    "connectedNodes" varchar(100) NULL,
    "InterLayerConnectionID" varchar(100) NULL
)
;

CREATE TABLE "InterLayerConnection_ThematicLayer"
(
    "connectedLayers" varchar(100) NULL,
    "InterLayerConnectionID" varchar(100) NULL
)
;

```

```

;

CREATE TABLE "Node"
(
    "geometry" geometry NULL,
    "NodeID" varchar(100) NOT NULL,
    "duality" varchar(100) NOT NULL,
    "DualSpaceLayerID" varchar(100) NOT NULL,
    connects jsonb NULL,
    CONSTRAINT "chk_Node_geometry_point" CHECK (
        "geometry" IS NULL
        OR GeometryType("geometry") = 'POINT'
    )
)
;

CREATE TABLE "PrimalSpaceLayer"
(
    "creationDate" timestamp without time zone NULL,
    "terminationDate" timestamp without time zone NULL,
    "PrimalSpaceLayerID" varchar(100) NOT NULL,
    "ThematicLayerID" varchar(100) NOT NULL
)
;

CREATE TABLE "ThematicLayer"
(
    "semanticExtension" boolean NOT NULL,
    "theme" "ThemeLayerValue" NOT NULL,
    "ThematicLayerID" varchar(100) NOT NULL,
    "IndoorFeaturesID" varchar(100) NOT NULL
)
;

/* Create Primary Keys, Indexes, Uniques, Checks */

ALTER TABLE "CellBoundary" ADD CONSTRAINT "PK_CellBoundary"
    PRIMARY KEY ("CellBoundaryID")
;

ALTER TABLE "CellSpace" ADD CONSTRAINT "PK_CellSpace"
    PRIMARY KEY ("CellSpaceID")
;

ALTER TABLE "DualSpaceLayer" ADD CONSTRAINT "PK_DualSpaceLayer"
    PRIMARY KEY ("DualSpaceLayerID")
;

ALTER TABLE "Edge" ADD CONSTRAINT "PK_Edge"
    PRIMARY KEY ("EdgeID")
;

ALTER TABLE "IndoorFeatures" ADD CONSTRAINT "PK_IndoorFeatures"
    PRIMARY KEY ("IndoorFeaturesID")
;

ALTER TABLE "InterLayerConnection" ADD CONSTRAINT "PK_InterLayerConnection"
    PRIMARY KEY ("InterLayerConnectionID")
;

ALTER TABLE "Node" ADD CONSTRAINT "PK_Node"
    PRIMARY KEY ("NodeID")
;

```

```

ALTER TABLE "PrimalSpaceLayer" ADD CONSTRAINT "PK_PrimalSpaceLayer"
PRIMARY KEY ("PrimalSpaceLayerID")
;

ALTER TABLE "ThematicLayer" ADD CONSTRAINT "PK_ThematicLayer"
PRIMARY KEY ("ThematicLayerID")
;

/* Create Foreign Key Constraints */

ALTER TABLE "CellBoundary" ADD CONSTRAINT "FK_CellBoundary_boundedBy"
FOREIGN KEY ("boundedBy") REFERENCES "CellSpace" ("CellSpaceID") ON DELETE No
Action ON UPDATE No Action
;

ALTER TABLE "CellBoundary" ADD CONSTRAINT "FK_CellBoundary_cellBoundaryMember"
FOREIGN KEY ("PrimalSpaceLayerID") REFERENCES "PrimalSpaceLayer"
("PrimalSpaceLayerID") ON DELETE No Action ON UPDATE No Action
;

ALTER TABLE "CellSpace" ADD CONSTRAINT "FK_CellSpace_cellSpaceMember"
FOREIGN KEY ("PrimalSpaceLayerID") REFERENCES "PrimalSpaceLayer"
("PrimalSpaceLayerID") ON DELETE No Action ON UPDATE No Action
;

ALTER TABLE "CellSpace" ADD CONSTRAINT "FK_CellSpace_duality"
FOREIGN KEY ("duality") REFERENCES "Node" ("NodeID") ON DELETE No Action ON
UPDATE No Action
;

ALTER TABLE "DualSpaceLayer" ADD CONSTRAINT "FK_DualSpaceLayer_dualSpace"
FOREIGN KEY ("ThematicLayerID") REFERENCES "ThematicLayer" ("ThematicLayerID")
ON DELETE No Action ON UPDATE No Action
;

ALTER TABLE "Edge" ADD CONSTRAINT "FK_Edge_duality"
FOREIGN KEY ("duality") REFERENCES "CellBoundary" ("CellBoundaryID") ON DELETE
No Action ON UPDATE No Action
;

ALTER TABLE "Edge" ADD CONSTRAINT "FK_Edge_DualSpaceLayer"
FOREIGN KEY ("DualSpaceLayerID") REFERENCES "DualSpaceLayer"
("DualSpaceLayerID") ON DELETE No Action ON UPDATE No Action
;

ALTER TABLE "Edge" ADD CONSTRAINT "FK_Edge_connects_node1"
FOREIGN KEY (connects_node1) REFERENCES "Node" ("NodeID") ON DELETE No Action ON
UPDATE No Action
;

ALTER TABLE "Edge" ADD CONSTRAINT "FK_Edge_connects_node2"
FOREIGN KEY (connects_node2) REFERENCES "Node" ("NodeID") ON DELETE No Action ON
UPDATE No Action
;

ALTER TABLE "InterLayerConnection" ADD CONSTRAINT "FK_InterLayerConnection_
layerConnections"
FOREIGN KEY ("IndoorFeaturesID") REFERENCES "IndoorFeatures"
("IndoorFeaturesID") ON DELETE No Action ON UPDATE No Action
;

```

```

ALTER TABLE "InterLayerConnection_CellSpace" ADD CONSTRAINT "FK_
InterLayerConnection_CellSpace_connectedCells"
FOREIGN KEY ("connectedCells") REFERENCES "CellSpace" ("CellSpaceID") ON DELETE
No Action ON UPDATE No Action
;

ALTER TABLE "InterLayerConnection_CellSpace" ADD CONSTRAINT "FK_
InterLayerConnection_CellSpace_InterLayerConnection"
FOREIGN KEY ("InterLayerConnectionID") REFERENCES "InterLayerConnection"
("InterLayerConnectionID") ON DELETE No Action ON UPDATE No Action
;

ALTER TABLE "InterLayerConnection_Node" ADD CONSTRAINT "FK_InterLayerConnection_
Node_connectedNodes"
FOREIGN KEY ("connectedNodes") REFERENCES "Node" ("NodeID") ON DELETE No Action
ON UPDATE No Action
;

ALTER TABLE "InterLayerConnection_Node" ADD CONSTRAINT "FK_InterLayerConnection_
Node_InterLayerConnection"
FOREIGN KEY ("InterLayerConnectionID") REFERENCES "InterLayerConnection"
("InterLayerConnectionID") ON DELETE No Action ON UPDATE No Action
;

ALTER TABLE "InterLayerConnection_ThematicLayer" ADD CONSTRAINT "FK_
InterLayerConnection_ThematicLayer_connectedLayers"
FOREIGN KEY ("connectedLayers") REFERENCES "ThematicLayer" ("ThematicLayerID")
ON DELETE No Action ON UPDATE No Action
;

ALTER TABLE "InterLayerConnection_ThematicLayer" ADD CONSTRAINT "FK_
InterLayerConnection_ThematicLayer_InterLayerConnection"
FOREIGN KEY ("InterLayerConnectionID") REFERENCES "InterLayerConnection"
("InterLayerConnectionID") ON DELETE No Action ON UPDATE No Action
;

ALTER TABLE "Node" ADD CONSTRAINT "FK_Node_duality"
FOREIGN KEY ("duality") REFERENCES "CellSpace" ("CellSpaceID") ON DELETE No
Action ON UPDATE No Action
;

ALTER TABLE "Node" ADD CONSTRAINT "FK_Node_DualSpaceLayer"
FOREIGN KEY ("DualSpaceLayerID") REFERENCES "DualSpaceLayer"
("DualSpaceLayerID") ON DELETE No Action ON UPDATE No Action
;

ALTER TABLE "PrimalSpaceLayer" ADD CONSTRAINT "FK_PrimalSpaceLayer_primalSpace"
FOREIGN KEY ("ThematicLayerID") REFERENCES "ThematicLayer" ("ThematicLayerID")
ON DELETE No Action ON UPDATE No Action
;

ALTER TABLE "ThematicLayer" ADD CONSTRAINT "FK_ThematicLayer_layers"
FOREIGN KEY ("IndoorFeaturesID") REFERENCES "IndoorFeatures"
("IndoorFeaturesID") ON DELETE No Action ON UPDATE No Action
;

/* Create Table Comments, Sequences for Autonumber Columns */

COMMENT ON TABLE "InterLayerConnection"
IS 'Describes the connection between two thematic layers. '
;

COMMENT ON COLUMN "InterLayerConnection"."typeOfTopoExpression"

```



```

    IS 'Describes the topological relationship between two layers (e.g. overlaps,
contains, etc.).'
;

COMMENT ON TABLE "ThematicLayer"
    IS 'A layer of specific theme aggregating a primal and/or a dual space of a
given environment.'
;

COMMENT ON COLUMN "ThematicLayer"."semanticExtension"
    IS 'Indicates whether semantic information is associated (true) or not (false)
to the primal space of the thematic layer.'
;

COMMENT ON COLUMN "ThematicLayer"."theme"
    IS 'Determines the theme of the layer (e.g topographic, logical, etc.).'
;

```

Listing B.1

B.2. SQL Schema of IndoorGML Navigation

```

/* ----- */
/* IndoorGML Navigation Schema Extension */
/* Apply AFTER IndoorGML_core.sql on the same database */
/* ----- */

/* Drop Navigation Tables */

DROP TABLE IF EXISTS "TransferSpace" CASCADE
;

DROP TABLE IF EXISTS "GeneralSpace" CASCADE
;

DROP TABLE IF EXISTS "ObjectSpace" CASCADE
;

DROP TABLE IF EXISTS "NonNavigableSpace" CASCADE
;

DROP TABLE IF EXISTS "NavigableBoundary" CASCADE
;

DROP TABLE IF EXISTS "NonNavigableBoundary" CASCADE
;

DROP TABLE IF EXISTS "NavigableSpace" CASCADE
;

DROP TABLE IF EXISTS "Route" CASCADE
;

DROP TABLE IF EXISTS "GeneralSpaceFunctionType" CASCADE
;

DROP TABLE IF EXISTS "LocomotionAccessType" CASCADE
;

```

```

DROP TABLE IF EXISTS "NavigableBoundaryFunctionType" CASCADE
;

DROP TABLE IF EXISTS "NetworkType" CASCADE
;

DROP TABLE IF EXISTS "TransferSpaceFunctionType" CASCADE
;

DROP TABLE IF EXISTS "TransferSpaceType" CASCADE
;

/* Drop ENUM Types */

DROP TYPE IF EXISTS "NavigableBoundaryFunctionType" CASCADE
;

DROP TYPE IF EXISTS "GeneralSpaceFunctionType" CASCADE
;

DROP TYPE IF EXISTS "LocomotionAccessType" CASCADE
;

DROP TYPE IF EXISTS "TransferSpaceCategoryType" CASCADE
;

DROP TYPE IF EXISTS "TransferSpaceFunctionType" CASCADE
;

/* Create ENUM Types */

CREATE TYPE "GeneralSpaceFunctionType" AS ENUM (
    'Administration',
    'Business, trade',
    'Education, training',
    'Recreation',
    'Laboratory',
    'Storage',
    'Security'
)
;

CREATE TYPE "LocomotionAccessType" AS ENUM (
    'Walking',
    'Flying',
    'Rolling',
    'Unspecified'
)
;

CREATE TYPE "NavigableBoundaryFunctionType" AS ENUM (
    'AnchorBoundary',
    'ConnectionBoundary'
)
;

CREATE TYPE "TransferSpaceFunctionType" AS ENUM (
    'ConnectionSpace',
    'AnchorSpace'
)
;

```

```

CREATE TYPE "TransferSpaceCategoryType" AS ENUM (
    'Door',
    'Window'
)
;

/* Create Navigation Tables */

CREATE TABLE "NavigableSpace"
(
    "locomotionType" "LocomotionAccessType" NOT NULL,
    "NavigableSpaceID" varchar(100) NOT NULL
)
;

CREATE TABLE "NonNavigableSpace"
(
    "NonNavigableSpaceID" varchar(100) NOT NULL
)
;

CREATE TABLE "GeneralSpace"
(
    "function" "GeneralSpaceFunctionType" NOT NULL,
    "GeneralSpaceID" varchar(100) NOT NULL
)
;

CREATE TABLE "TransferSpace"
(
    "function" "TransferSpaceFunctionType" NOT NULL,
    "category" "TransferSpaceCategoryType" NOT NULL,
    "TransferSpaceID" varchar(100) NOT NULL
)
;

CREATE TABLE "ObjectSpace"
(
    "description" text NULL,
    "containedFeatures" integer NULL,
    "ObjectSpaceID" varchar(100) NOT NULL
)
;

CREATE TABLE "NavigableBoundary"
(
    "NavigableBoundaryFunction" "NavigableBoundaryFunctionType" NOT NULL,
    "boundaryOrientation" boolean NULL,
    "NavigableBoundaryID" varchar(100) NOT NULL
)
;

CREATE TABLE "NonNavigableBoundary"
(
    "NonNavigableBoundaryID" varchar(100) NOT NULL
)
;

CREATE TABLE "Route"
(
    "creationDate" timestamp without time zone NULL,
    "RouteID" varchar(100) NOT NULL,
    "routeNode" jsonb NULL,

```

```

"routeEdge" jsonb NULL
)
;

/* Primary Keys */

ALTER TABLE "NavigableSpace" ADD CONSTRAINT "PK_NavigableSpace"
PRIMARY KEY ("NavigableSpaceID")
;

ALTER TABLE "NonNavigableSpace" ADD CONSTRAINT "PK_NonNavigableSpace"
PRIMARY KEY ("NonNavigableSpaceID")
;

ALTER TABLE "GeneralSpace" ADD CONSTRAINT "PK_GeneralSpace"
PRIMARY KEY ("GeneralSpaceID")
;

ALTER TABLE "TransferSpace" ADD CONSTRAINT "PK_TransferSpace"
PRIMARY KEY ("TransferSpaceID")
;

ALTER TABLE "ObjectSpace" ADD CONSTRAINT "PK_ObjectSpace"
PRIMARY KEY ("ObjectSpaceID")
;

ALTER TABLE "NavigableBoundary" ADD CONSTRAINT "PK_NavigableBoundary"
PRIMARY KEY ("NavigableBoundaryID")
;

ALTER TABLE "NonNavigableBoundary" ADD CONSTRAINT "PK_NonNavigableBoundary"
PRIMARY KEY ("NonNavigableBoundaryID")
;

ALTER TABLE "Route" ADD CONSTRAINT "PK_Route"
PRIMARY KEY ("RouteID")
;

/* Foreign Keys */

ALTER TABLE "NavigableSpace" ADD CONSTRAINT "FK_NavigableSpace_CellSpace"
FOREIGN KEY ("NavigableSpaceID") REFERENCES "CellSpace" ("CellSpaceID") ON
DELETE No Action ON UPDATE No Action
;

ALTER TABLE "NonNavigableSpace" ADD CONSTRAINT "FK_NonNavigableSpace_CellSpace"
FOREIGN KEY ("NonNavigableSpaceID") REFERENCES "CellSpace" ("CellSpaceID") ON
DELETE No Action ON UPDATE No Action
;

ALTER TABLE "GeneralSpace" ADD CONSTRAINT "FK_GeneralSpace_NavigableSpace"
FOREIGN KEY ("GeneralSpaceID") REFERENCES "NavigableSpace" ("NavigableSpaceID")
ON DELETE No Action ON UPDATE No Action
;

ALTER TABLE "TransferSpace" ADD CONSTRAINT "FK_TransferSpace_NavigableSpace"
FOREIGN KEY ("TransferSpaceID") REFERENCES "NavigableSpace"
("NavigableSpaceID") ON DELETE No Action ON UPDATE No Action
;

ALTER TABLE "ObjectSpace" ADD CONSTRAINT "FK_ObjectSpace_NonNavigableSpace"
FOREIGN KEY ("ObjectSpaceID") REFERENCES "NonNavigableSpace"
("NonNavigableSpaceID") ON DELETE No Action ON UPDATE No Action

```

```

;

ALTER TABLE "NavigableBoundary" ADD CONSTRAINT "FK_NavigableBoundary_
CellBoundary"
    FOREIGN KEY ("NavigableBoundaryID") REFERENCES "CellBoundary"
    ("CellBoundaryID") ON DELETE No Action ON UPDATE No Action
;

ALTER TABLE "NonNavigableBoundary" ADD CONSTRAINT "FK_NonNavigableBoundary_
CellBoundary"
    FOREIGN KEY ("NonNavigableBoundaryID") REFERENCES "CellBoundary"
    ("CellBoundaryID") ON DELETE No Action ON UPDATE No Action
;

```

Listing B.2



ANNEX C (NORMATIVE) CORE CODE LIST TYPES



ANNEX C (NORMATIVE) CORE CODE LIST TYPES

The following PostgreSQL ENUM types represent IndoorGML core code lists:

- ThemeLayerValue for the ThematicLayer.theme column;
- TopoExpressionValue for the InterLayerConnection.typeOfTopoExpression column.

```
CREATE TYPE "ThemeLayerValue" AS ENUM (  
    'Virtual',  
    'Physical',  
    'Tags',  
    'Unknown'  
);  
  
CREATE TYPE "TopoExpressionValue" AS ENUM (  
    'Contains',  
    'Overlaps',  
    'Equals',  
    'Within',  
    'Crosses',  
    'Other'  
);
```

Listing C.1 — A SQL schema of the core code list types



ANNEX D (NORMATIVE) NAVIGATION CODE LIST TYPES

D

ANNEX D (NORMATIVE) NAVIGATION CODE LIST TYPES

The following PostgreSQL ENUM types represent IndoorGML navigation code lists:

- `LocomotionAccessType` for the `NavigableSpace.locomotionType` column;
- `GeneralSpaceFunctionType` for the `GeneralSpace.function` column;
- `TransferSpaceFunctionType` for the `TransferSpace.function` column;
- `TransferSpaceCategoryType` for the `TransferSpace.category` column;
- `NavigableBoundaryFunctionType` for the `NavigableBoundary.NavigableBoundaryFunction` column.

```
CREATE TYPE "GeneralSpaceFunctionType" AS ENUM (  
    'Administration',  
    'Business, trade',  
    'Education, training',  
    'Recreation',  
    'Laboratory',  
    'Storage',  
    'Security'  
);  
  
CREATE TYPE "LocomotionAccessType" AS ENUM (  
    'Walking',  
    'Flying',  
    'Rolling',  
    'Unspecified'  
);  
  
CREATE TYPE "NavigableBoundaryFunctionType" AS ENUM (  
    'AnchorBoundary',  
    'ConnectionBoundary'  
);  
  
CREATE TYPE "TransferSpaceFunctionType" AS ENUM (  
    'ConnectionSpace',  
    'AnchorSpace'  
);  
  
CREATE TYPE "TransferSpaceCategoryType" AS ENUM (  
    'Door',  
    'Window'
```

);

Listing D.1 — A SQL schema of the navigation code list types



ANNEX E (INFORMATIVE) REVISION HISTORY

E

ANNEX E (INFORMATIVE) REVISION HISTORY

DATE	RELEASE	EDITOR	PRIMARY CLAUSES MODIFIED	DESCRIPTION
2026-06-17	0.1	Zhiyong Wang	all	initial version derived from SQL generation workflow document
2026-06-18	0.2	Zhiyong Wang	5, 6, 7, 8, annex B	align schema with ENUM types, jsonb connects/route arrays, and EA table naming
2026-07-02	0.3	Zhiyong Wang	5, 6	add Part 2c-specific symbols and abbreviated terms table (clause 5); replace vague figure references with numbered cross-references in clause 6.1.1 and remove redundant follow-up sentences; rebuild document.pdf and document.html
2026-07-30	0.4	Zhiyong Wang	5, 6, 7	flatten geometry DataTypes to path-named PostGIS columns; encode ExternalReference DataTypes as Postgre SQL composite types; PascalCase table names; edition 0.0.9
2026-08-16	0.5	Zhiyong Wang	front matter, 5, 6, 7, 8, annex C	update document number to 26-044; replace Edge . connects jsonb with connects_node1 / connects_node2 foreign keys to Node; move Core code list types from Clause 7.1 to Annex C; define ObjectSpace . Containedfeatures as integer; edition 0.5.0
2026-08-17	0.5	Zhiyong Wang	annex A	redefine Abstract Test Suite with one abstract test per requirement using Metanorma conformance_class / abstract_test format aligned with IndoorJSON Part II
2026-08-17	0.5	Zhiyong Wang	5, 6, 7, 8, annex A/C, schema	align SQL attribute names/casing with UML (e.g. is Directed, creationDate, cellSpaceGeom_Geometry2D, boundedBy); edition 0.5.0
2026-08-17	0.5	Zhiyong Wang	7, 8, annex A, schema	mark UML-mandatory attributes and associations as NOT NULL (e.g. isLogical, isDirected, Pol, weight, connects_node1/2, duality)
2026-08-17	0.5	Zhiyong Wang	5, 7, schema	document PostGIS GM_Solid encoding as POLYHEDRALSURFACE Z / TIN Z with CellSpace Geometry3D CHECK constraint

DATE	RELEASE	EDITOR	PRIMARY CLAUSES MODIFIED	DESCRIPTION
2026-08-17	0.5	Zhiyong Wang	5, 7, annex A/B, schema	add permitted geometry type table, geometry-type CHECK constraints for CellSpace, CellBoundary, Node and Edge, and corresponding normative requirement parts and abstract tests
2026-08-17	0.5	Zhiyong Wang	abstract	revise Abstract to describe PostgreSQL as an object-relational DBMS, store and maintain models, and access via API or web services
2026-08-17	0.5	Zhiyong Wang	3, 5, 7, 8	remove IndoorJSON cross-references and JSON-FG requirement-class inheritance; keep general JSON and GeoJSON bibliography entries
2026-08-17	0.5	Zhiyong Wang	3, 4	add data type to Terms and Definitions and cite ISO 19103



BIBLIOGRAPHY





BIBLIOGRAPHY

- [1] H. Butler, M. Daly, A. Doyle, S. Gillies, S. Hagen, T. Schaub: IETF RFC 7946, *The GeoJSON Format*. RFC Publisher (2016). <https://www.rfc-editor.org/info/rfc7946>.
- [2] T. Bray (ed.): IETF RFC 8259, *The JavaScript Object Notation (JSON) Data Interchange Format*. RFC Publisher (2017). <https://www.rfc-editor.org/info/rfc8259>.
- [3] The PostGIS Development Group: **PostGIS Manual**, <https://postgis.net/docs/>
- [4] Ben-Kiki, O., Evans, C., Ingy döt Net: **YAML Ain't Markup Language**, <https://yaml.org/>